大语言模型的训练

第 9 章

训练显存与并行 · 混合并行策略 · 分布式训练系统

著者
邓杭　杨耀东

教材
《大语言模型：训练、精调与对齐》

出版
清华大学出版社



本章目录

9.1 **大语言模型训练** 第 3 页
三大挑战、训练框架、显存开销与通信基础

9.2 **混合并行策略** 第 24 页
四种并行及其正交组合

9.3 **分布式训练系统** 第 61 页
DeepSpeed 零冗余优化器与显存分区

9.4 **小结** 第 75 页
并行算法的取舍与系统视角

9.5 **附录：专家并行*** 第 79 页
MoE 的专家部署与两次全交换

9.6 **附录：训练-推理解耦架构*** 第 84 页
RL 训练的采样部署、参数同步与异步推理

阅读建议 先从显存与并行维度理解训练瓶颈（9.1），再读混合并行与系统实现（9.2–9.3）。

9.1

大语言模型训练

训练规模的增长把问题从“模型能不能学”推到“系统能不能撑住”：显存、带宽与可靠性共同决定训练能否跑完。

本节内容

9.1.1 介绍

9.1.2 分布式训练框架

9.1.3 训练目标

9.1.4 设备逻辑架构

9.1.5 分布式通信操作

大语言模型的训练规模

随着预训练模型的参数规模与训练语料库持续增长，模型训练对计算资源的需求急剧上升，通常需要庞大的 GPU 集群进行长时间持续训练。

16 000

H100-80GB GPU，Llama-3 预训练所用的集群规模

54

天，一次预训练持续的时间

3

核心挑战：可扩展性、高效性、可靠性

> **指标** 本章重点聚焦提升训练软件框架的高效性，其核心衡量指标是吞吐率：平均每秒每 GPU 处理的词元数量。该指标值越高，表明运行效率越高。

吞吐率是算法设计、计算优化、网络通信、存储管理和训练框架实现等多方面因素共同作用的结果，因此需要逐层拆解系统的瓶颈。

训练的三大挑战

大规模语言模型训练面临的核心挑战在于基础设施的可扩展性、高效性和可靠性。

挑战 1

可扩展性

要求基础设施和系统能够无缝扩展到更多 GPU 或其他计算设备上，同时保持训练的准确性和模型精度。

涉及分布式训练系统对硬件网络、存储和训练框架的协同设计。

挑战 2

高效性

旨在最大化集群资源的利用率，通常使用模型 FLOPs 利用率（MFU）作为衡量指标。

需要精心设计并行算法、优化计算资源利用、最小化通信开销，并实施高效的存储管理。

挑战 3

可靠性

源于预训练过程往往持续数周甚至数月，必须确保训练过程中硬件和软件的稳定性。

需要设计有效的容错机制，并在训练因故中断后能迅速恢复。

三个挑战相互牵制：并行度越高越容易扩展，但通信开销上升会压低 MFU；而规模越大、运行越久，故障概率也越高。

本章结构

预训练中常用的分布式训练策略是混合并行训练，其关键组成部分包括数据并行、张量并行、流水线并行和序列并行。

逻辑设备

9.1.4 节

集群的通信层级

通信操作

9.1.5 节

集合通信原语

混合并行

9.2 节

DP / TP / PP / CP

ZeRO

9.3 节

显存冗余消除

- DeepSpeed 框架中的 ZeRO 技术能显著提升集群上的存储效率，见 9.3 节。
- 对于集成了 MoE 的网络结构，还需引入专家并行，见 9.5 节附录。
- 基于强化学习的训练算法需要训练系统与推理系统并行部署，见 9.6 节附录。

效率优化技术

技术 1

梯度检查点

前向计算时仅保留部分激活值，反向传播时重新计算丢弃的部分，显著降低显存消耗。

技术 2

混合精度训练

结合使用不同精度的浮点数，在保持数值稳定的同时加速计算。

技术 3

低比特训练

使用 FP8 等超低精度格式，能极大减少计算量和通信量。

技术 4

计算内核优化

通过定制高性能计算内核（如融合算子）进一步提升训练速度。

DeepSeek-V3 的三个工程选择说明了这些技术的组合方式：双向流水线并行、计算-通信重叠掩盖专家并行的通信延迟，以及在其大型模型训练中应用的 8 位精度训练技术。

> **注** 计算-通信重叠是贯穿全章的主线：通信本身的耗时难以消除，但可以把通信藏在计算背后（详见 9.2 节的 DualPipe 与 9.5 节）。

分布式训练框架的分类

在大规模深度学习训练中，常用的分布式训练框架包括 PyTorch、DeepSpeed、Megatron-LM 和 Ray，它们各自负责训练栈的不同层次。

框架 1

PyTorch.Distributed

支持在多台机器和多个 GPU 上进行训练，提供通信操作接口和进程组管理功能，支持数据并行策略。

框架 2

DeepSpeed

分布式数据并行的优化实现，核心优势在于 ZeRO 优化器系列技术，可对参数、梯度、优化器状态分片或卸载。

框架 3

Megatron-LM

定制优化了 Transformer 架构，深度集成张量并行与流水线并行，通过算子切分规则与通信模式分布参数。

框架 4

Ray

通用的分布式计算框架，提供灵活的 API 与任务调度、资源管理，可协调训练与在线采样两类任务。

PyTorch.Distributed

PyTorch.Distributed 是 PyTorch 中用于支持分布式训练的模块，它通过抽象底层通信接口，支持多种后端，并通常基于单程序多数据（SPMD）范式进行编程。

- 提供通信操作接口和进程组管理功能，支持数据并行策略。
- 提供 All-Reduce、All-Gather、All-to-All 等通信原语，实现不同进程间的数据共享与同步。

SPMD

单程序多数据

所有进程执行同一份程序代码，只是各自处理不同的数据分片，并通过 rank 区分身份。

这种范式让并行算法的表达与设备数量解耦：同一段代码在 8 卡与 800 卡上都能运行，只需改变通信组的大小。

> **注** PyTorch.Distributed 只提供“通信怎么做”，不决定“如何切分模型”。张量并行、流水线并行的切分规则由 Megatron-LM 这类框架在其之上实现。

四种框架的定位

框架	主导方	核心机制	在训练栈中的位置
PyTorch.Distributed	PyTorch 社区	集合通信原语、进程组、SPMD	通信底座，数据并行的实现基础
DeepSpeed	微软	ZeRO：参数、梯度、优化器状态分片与卸载	数据并行的显存优化层
Megatron-LM	NVIDIA	张量并行、流水线并行的算子切分与通信模式	模型并行的实现层
Ray	加州大学伯克利分校	通用任务调度、资源管理、参数服务器	跨任务的编排层，用于 RL 训练与采样

四者的关系不是替代而是分工：Megatron-LM 解决“模型放不下”，DeepSpeed 解决“状态太冗余”，Ray 解决“任务怎么编排”，PyTorch.Distributed 提供底层的通信能力。

在 LLM 强化学习训练场景（如 PPO 和 GRPO）中，常需在线采样。Ray 能有效协调训练和在线采样任务，并通过参数服务器模式或共享存储机制管理参数状态、保障分布式环境下的同步一致性。

分布式训练框架概览

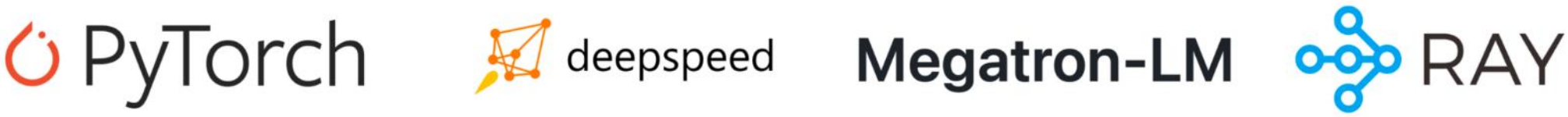


图 9.1　常用的分布式训练框架

框架之外还有两个补充工具，二者的共同思路都是用 Triton 重写关键算子。

算子层

Liger Kernel

针对大语言模型训练优化的 Triton 内核集合，重写 RMSNorm、RoPE、SwiGLU、CrossEntropy 等常见训练算子，显著提升多 GPU 训练的吞吐量并降低内存占用。

微调层

Unsloth

面向高效微调的开源框架，使用 Triton 手工重写关键 PyTorch 算子，把计算密集型操作直接优化为高性能 GPU 内核，消除计算冗余。

训练显存的组成

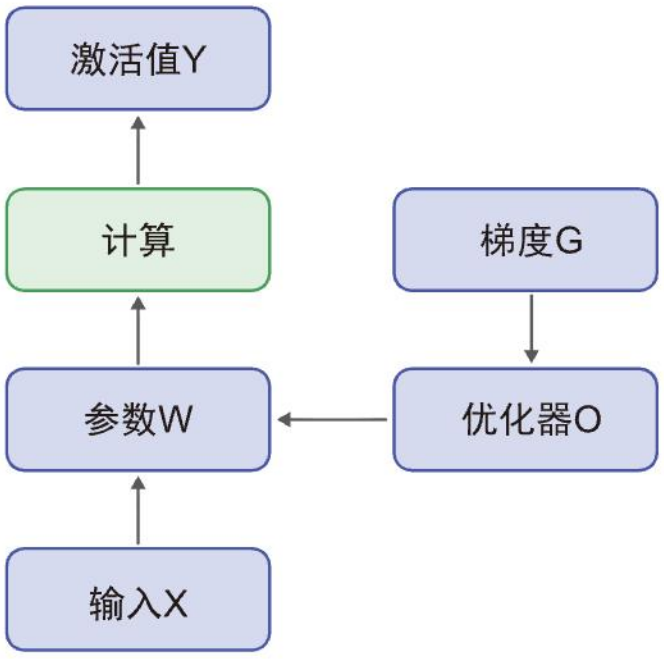


图 9.2　语言模型训练过程存储占用

主流大语言模型通常采用纯解码器架构的 Transformer 模型，例如 GPT 和 Llama 系列。训练时通常使用 Adam 优化器。

- 前向传播过程会保留**激活值**，用于反向传播时的梯度计算。
- Adam 优化器自身维护的**动量**和**方差**状态也会占用显存。
- 此外还需保存**模型参数**与**梯度**，四项共同构成常驻开销。

分布式的必要性

压力 1

显存放不下

训练所需显存通常远超单设备的存储容量，因此需要将数据或模型切分到多个设备上进行分布式训练。

压力 2

通信停不住

设备间需要通过通信操作同步参数或中间数据，这些通信可能导致计算单元空闲等待，降低训练效率。

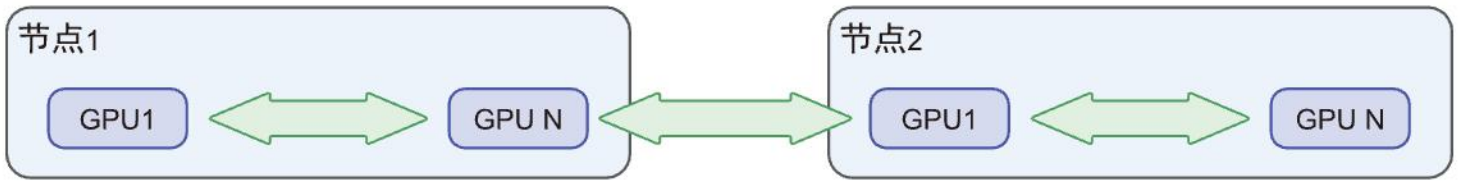


图 9.3　通信操作导致设备计算资源空闲，降低训练 MFU 效率

提升 MFU 有两条路：优化计算与通信的重叠；把数据分发到更多设备以提高并行度。通常采用混合并行策略，同时减少显存占用和通信开销。

分布式训练中的通信操作（如 All-Gather）有时也会产生需要临时存储的中间数据，这部分显存开销同样要计入预算。

集群的三级结构

在一个计算集群中，计算设备的最基本单元是 GPU。多个 GPU 组成一个计算节点，多个计算节点则构成整个集群；除计算设备外，网络组件也是集群的关键部分。

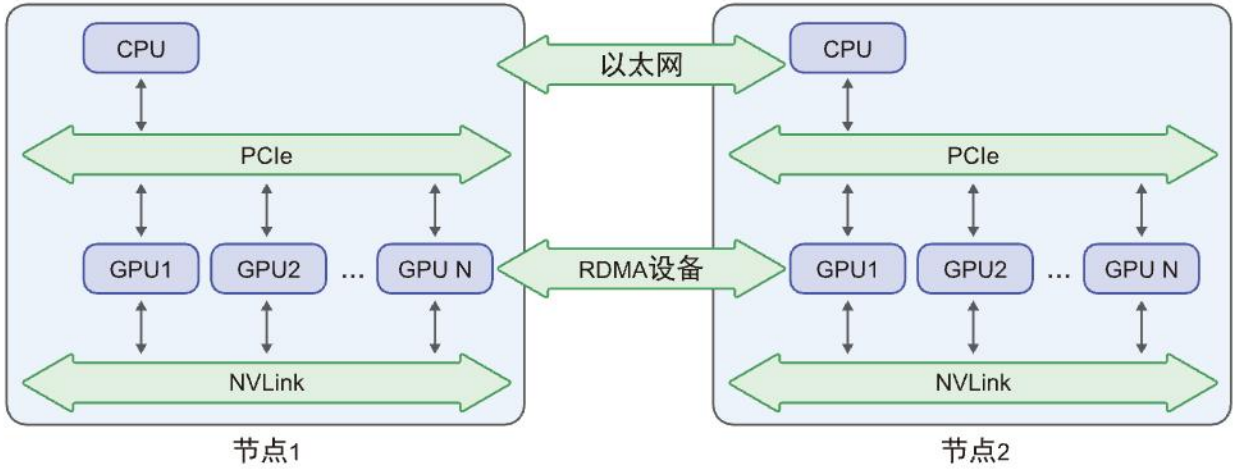


图 9.4　多节点多 GPU 集群设备间的通信逻辑架构图

GPU

计算的基本单元

→

节点

多块 GPU + 节点内互连

→

集群

多节点 + 节点间网络

三种通信链路

第 1 层 · CPU 与 GPU

PCIe 总线

数据或模型从存储设备传输到 GPU 即通过此通道。

这一层决定了数据加载的上限，数据管道供不上时 GPU 会空转。

第 2 层 · 节点内 GPU 与 GPU

PCIe / NVLink / NVSwitch

基础通信通过 PCIe 实现（如 GPU Direct P2P）；高速互连可采用 NVLink 提供高带宽、低延迟的点对点直接连接。

NVSwitch 作为交换芯片，为连接在其上的多个 GPU 提供全带宽、无阻塞的通信能力。

第 3 层 · 节点间 GPU 通信

RDMA

传统节点间通信基于以太网与 TCP/IP 协议栈，需要操作系统和 CPU 深度参与，延迟较高。

RDMA 允许一台计算机直接访问另一台计算机的内存，无需操作系统介入，显著降低延迟、提高吞吐量。

> **实现** RDMA 可通过专用的 InfiniBand 网络，或支持 RoCE（RDMA over Converged Ethernet）的以太网实现高速互联。

带宽的层级差异直接决定了并行算法的部署位置：通信密集型的任务（如张量并行）尽量限制在节点内完成；通信频率较低的任务（如数据并行，其主要通信发生在梯度同步阶段）则适合跨节点部署。

通信的初始化

网络拓扑

初始化 Mesh

通信群组

划分参与通信的进程

标识

本地设备号、IP 与端口

通信库

MPI / NCCL / GLOO

- 系统中的每个设备或进程拥有一个固定的本地设备号，并在网络层面通过唯一的 IP 地址和端口号进行标识。
- 通过通信库（如 MPI、NCCL 和 GLOO 等后端）初始化后，进程间可利用逻辑进程标识符（如 rank）或网络地址进行点对点通信。
- 点对点通信支持阻塞或非阻塞模式，是构造集合通信的基础。

rank 是并行算法中的“坐标”：数据切分、参数分区、环形拓扑中的前驱与后继，最终都要落到某个 rank 上。

阻塞通信与异步通信

阻塞式通信（同步通信）

要求发送方和接收方必须均完成数据发送和接收操作后，才能各自继续执行后续操作。

语义简单、易于推理，但设备在等待对端时完全空闲。

非阻塞式通信（异步通信）

允许发送方在发起发送操作后立即返回，继续执行其他计算任务，而无需等待接收方完成接收；接收方同样可立即返回，待数据传输完成后再处理。

异步通信有助于减少通信等待时间，提高计算资源利用率。

> **作用** 异步通信在异步数据并行和零气泡流水线并行等优化技术中起到关键作用。

本章后续的所有“重叠”技巧——无论是流水线的 1F1B、ZeroBubble 的 1F1B1W，还是 DeepSeek-V3 的 DualPipe——本质上都是把阻塞式通信改造成可与计算并行推进的异步通信。

集合通信原语

除点对点通信外，分布式训练广泛依赖一组标准的集合通信操作，包括广播、散播、全收集、全规约和全交换等。这些操作在通信群组内所有进程间协调执行，底层实现通常基于点对点通信原语，且是去中心化的。

操作	数据流向	典型用途
Broadcast	根进程的一份数据复制到全部设备	分发初始参数、配置
Scatter	根进程的 N 个子块分发给 N 个设备	切分数据或参数
Gather / All-Gather	各设备的数据块汇聚到根 / 汇聚到全部	拼回完整输出、ZeRO-3 取参数
Reduce / All-Reduce	规约结果存于根 / 全部设备各得一份	梯度同步
All-to-All	每台设备向每台设备发送不同的块	专家并行的派发与合并

熟练掌握这些常规集合通信操作，是实现更复杂分布式训练算法的基础：数据并行只用到 All-Reduce，ZeRO 用到 Reduce-Scatter 与 All-Gather，专家并行用到 All-to-All。

广播与散播

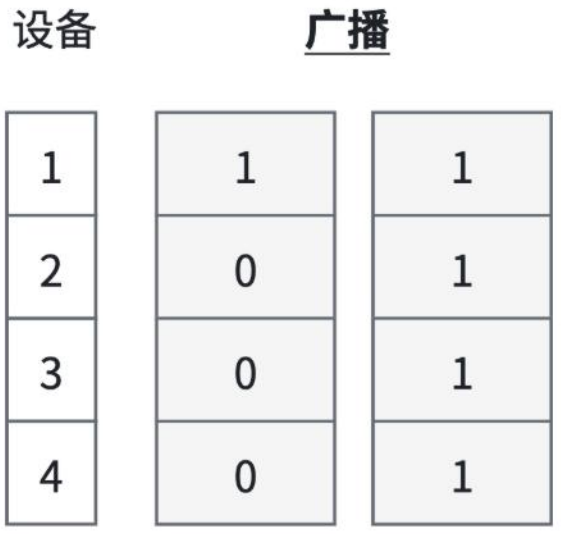


图 9.5(a)　广播



图 9.5(b)　散播

广播 Broadcast

设组内共有 N 个设备，根进程 i 将其数据 $\boldsymbol{X}$ 发送给其他 $N-1$ 个设备，最终所有设备均持有 $\boldsymbol{X}$ 的完整副本。

散播 Scatter

根进程将数据均等拆分为 N 个子块 $\boldsymbol{X} \to (\boldsymbol{X}_1, \boldsymbol{X}_2, \cdots, \boldsymbol{X}_N)$，第 j 块发送至设备 j；各设备接收的数据块互不相同。

图 《大语言模型：训练、精调与对齐》图 9.5；第 9.1.5 节

收集与全收集

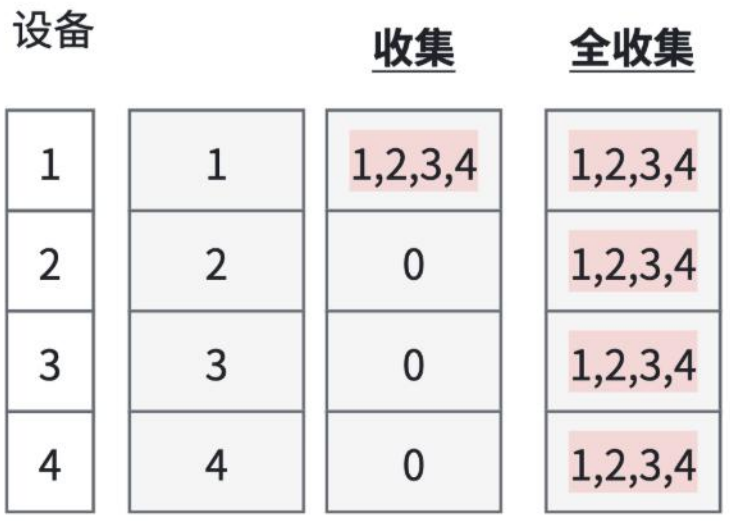


图 9.5(c)　收集和全收集

$$\boldsymbol{X} \leftarrow \texttt{Gather}(\boldsymbol{X}_1, \boldsymbol{X}_2, \cdots, \boldsymbol{X}_N)$$

$$\boldsymbol{X} \leftarrow [\boldsymbol{X}_1, \boldsymbol{X}_2, \cdots, \boldsymbol{X}_N], \quad \boldsymbol{X} \in \mathbb{R}^d$$

Gather 把 N 个数据块汇聚到根进程；All-Gather 让每个设备都得到全部数据块并按维度拼接。

收集 Gather

散播的逆操作

各设备把本地块发往根进程 i，根进程组合出完整数据集。

全收集 All-Gather

人人都有完整数据

设备 i 持有 $\boldsymbol{X}_i \in \mathbb{R}^{d/N}$，全收集后所有设备都获得全部数据块。

规约类原语



(a) 规约（以平均为例）　(b) 全规约（以平均为例）　(c) 规约-散播

图 9.6　分布式集合通信-2

$$\tilde{\boldsymbol{X}} = \mathrm{op}(\boldsymbol{X}_1, \boldsymbol{X}_2, \cdots, \boldsymbol{X}_N), \quad \tilde{\boldsymbol{X}} \in \mathbb{R}^d$$

$$\hat{\boldsymbol{X}}^{(j)} = \mathrm{op}(\boldsymbol{X}_1^{(j)}, \boldsymbol{X}_2^{(j)}, \cdots, \boldsymbol{X}_N^{(j)}), \quad \hat{\boldsymbol{X}}^{(j)} \in \mathbb{R}^{d/N}$$

op 表示规约运算符，如求和、求平均、取最大值（如 Mean）。式 1 为 Reduce，结果存于根设备；式 2 为 Reduce-Scatter，设备 j 获得第 j 个结果块。

全交换

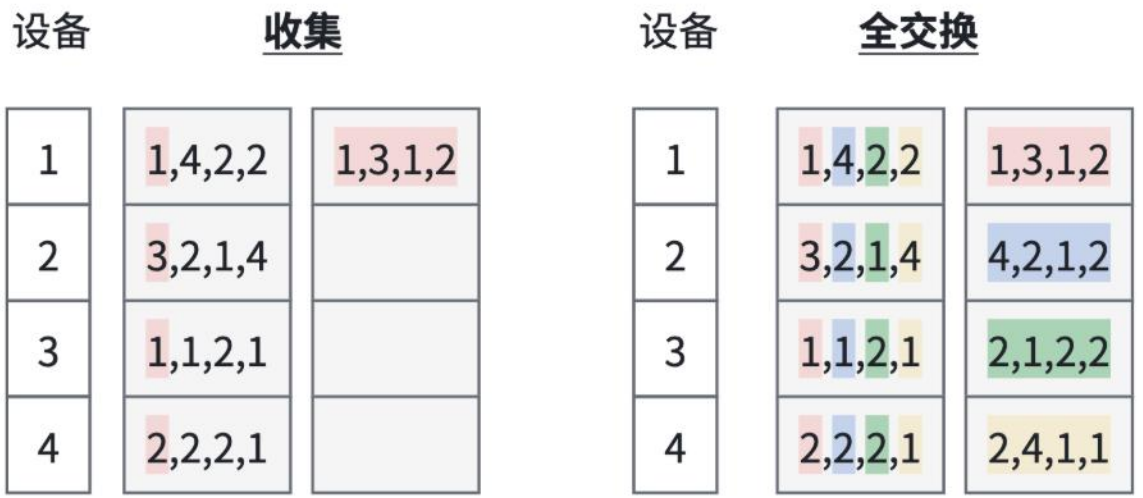


图 9.7　分布式集合通信-3

- 通信组内 N 个设备各自将数据划分为 N 个块，数据块大小可以不一致；设备 i 的块 $\boldsymbol{X}_i^{(j)}$ 发送至设备 j。
- 操作完成后，设备 j 接收来自所有设备的第 j 块数据 $\{\boldsymbol{X}_1^{(j)}, \boldsymbol{X}_2^{(j)}, \cdots, \boldsymbol{X}_N^{(j)}\}$。
- 在大语言模型中，全交换常用于专家并行的派发-合并过程，此时面临交换数据量不均导致的负载均衡难题。

图 《大语言模型：训练、精调与对齐》图 9.7；第 9.1.5 节

全交换与全收集的区别

朴素实现：轮流当根

各设备轮流作为根节点，依次收集来自其他设备的不同数据。每轮单次收集中，设备角色互斥（一个接收、多个发送），需同步完成 N 轮。

N 轮串行，时延随设备数线性增长。

高效实现：一个周期完成

在一个集合通信周期内完成所有数据交换，该周期内每个设备同时作为发送方和接收方。

发送与接收并行，链路利用率显著提高。

操作	发送逻辑	结果
All-Gather	广播：每个设备向所有设备发送**相同**数据	所有设备持有完整副本
All-to-All	散播：每个设备向不同目标发送**不同**数据	每个设备持有各来源的对应块

这条区别解释了两种原语在训练中的分工：All-Gather 用于“补齐”被切分的参数或激活，All-to-All 用于“换手”——把词元送到持有其专家的设备上。

9.2

混合并行策略

核心挑战在于执行远超单个设备显存容量的计算任务，同时高效利用集群资源；四种并行维度彼此正交，可以组合成 4D 并行。

本节内容

9.2.1 数据并行

9.2.2 张量并行

9.2.3 流水线并行

9.2.4 上下文并行

9.2.5 本节小结

四种并行方式

示例使用 2 个计算设备和 4 个数据样本，通过从模型参数和数据两个维度进行不同方式的切分，实现了不同的并行方法。

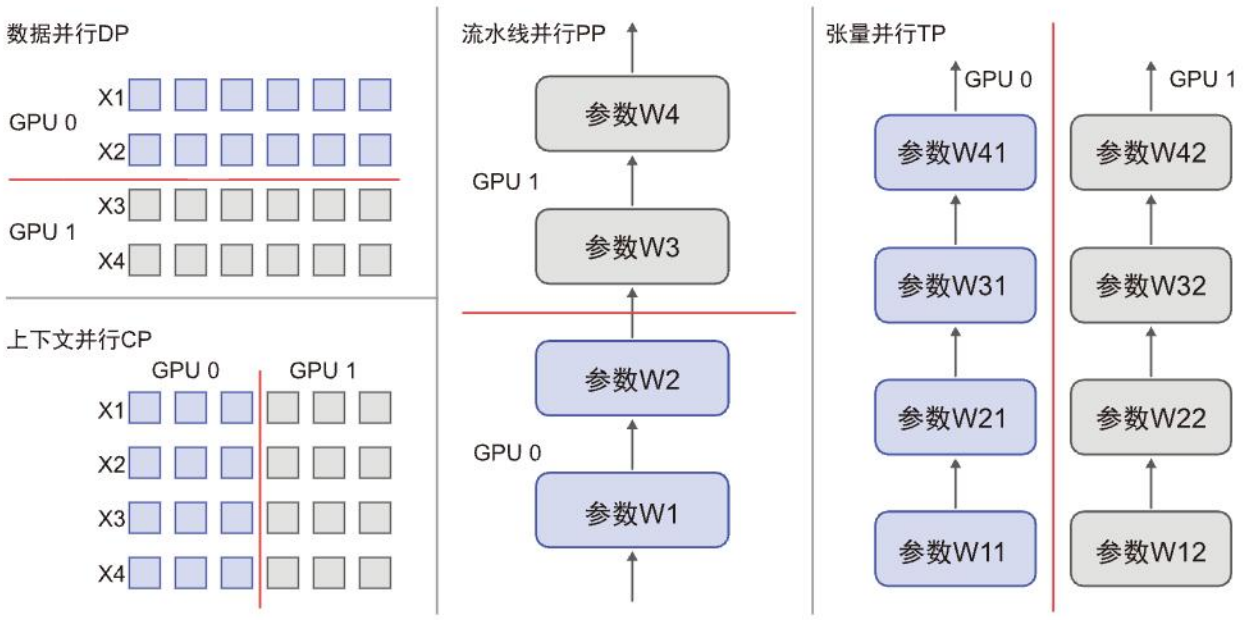


图 9.8　合并行策略（4D 并行）示例

左上

数据并行：样本均分，参数完整复制。

中间

流水线并行：按层深度切分，参数按层分属两卡。

左下

上下文并行：按序列长度切分，各自计算块注意力。

最右

张量并行：参数矩阵切分，每卡只算一部分。

切分的代价

分布式训练的核心目标是在有限的计算资源下，于单位时间内处理更多的数据量。而模型切分的主要目的，是将庞大的计算模型分解以适应有限的 GPU 显存，从而避免因显存不足而无法运行。

> **关键** 模型切分本身并不会减少训练任务的总计算量；相反，它会引入额外的通信、存储和计算开销。

收益

显存可容纳

把参数、梯度、优化器状态或激活值分摊到多张卡上，使单卡放不下的模型可以训练。

代价 1

额外通信

切分边界上需要交换中间结果或梯度，通信可能让计算单元空转。

代价 2

额外存储与计算

通信需要缓冲区，某些切分方式还会带来冗余计算与参数收集开销。

Llama-3 的 4D 并行

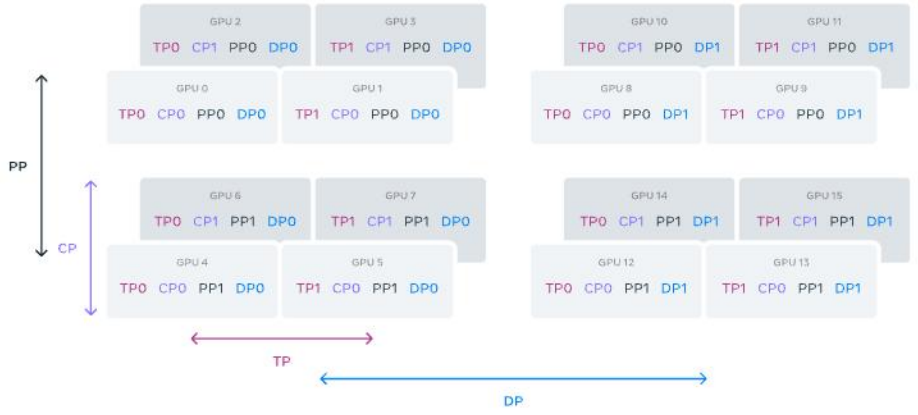


图 9.9　Llama-3 的 4D 并行训练策略

当每个维度的并行切分因子均为 2 时，四个维度的并行策略组合最多可分布在 16 个 GPU 上，即 $2^4 = 16$。

$$N_{\text{GPU}} = N_{\text{DP}} \times N_{\text{TP}} \times N_{\text{PP}} \times N_{\text{CP}} = 2 \times 2 \times 2 \times 2 = 16$$

各维度的切分因子相乘得到总设备数。通过灵活配置不同维度的并行策略，可以在混合并行框架下实现高效训练。

混合并行的核心目标在于提升吞吐量和设备利用率：通信密集的维度放节点内，通信稀疏的维度跨节点。

图 《大语言模型：训练、精调与对齐》图 9.9；第 9.2 节

数据并行

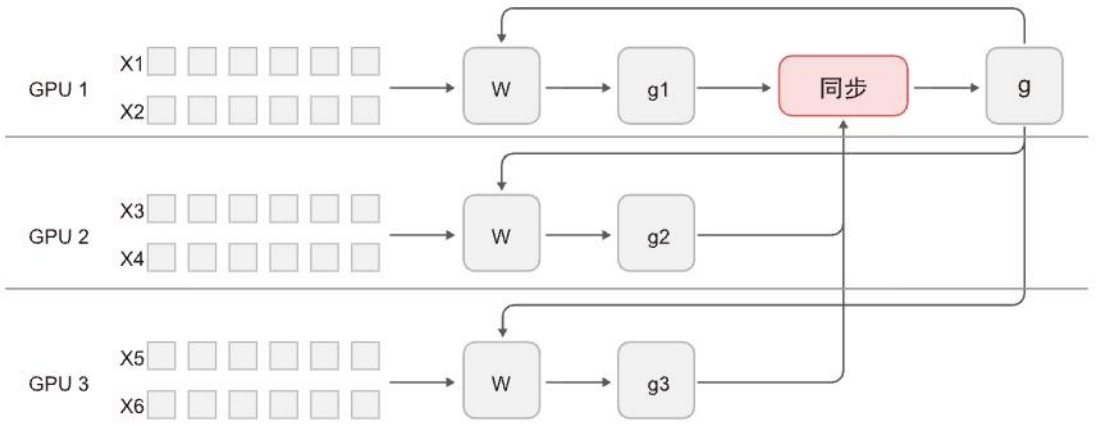


图 9.10　数据并行的梯度更新流程

数据并行在多个计算设备上加载相同参数的模型副本，每个设备处理不同批次的输入数据，独立计算梯度。

- 输入数据集 $\{\boldsymbol{x}_1, \boldsymbol{x}_2, \cdots, \boldsymbol{x}_6\}$ 被分配到 3 个 GPU 上，每个 GPU 上的模型参数 $\boldsymbol{W}$ 初始相同。
- 各设备独立前向计算损失，再根据独立的损失进行梯度反向传播。
- 随后规约各设备的梯度，同步更新参数，保持所有设备上的参数一致。

图中指定 GPU0 作为参数服务器进行梯度同步。通过增加计算设备，数据并行可以并行处理更多数据，但会引入梯度同步所需的通信开销。

梯度同步的时机

梯度同步可以在整个反向传播过程完成后一次性进行，也可以在反向传播过程中分层进行。

批量更新

反向传播全部完成后统一同步一次梯度。

通信次数少，可以合并成大消息、充分利用带宽。

代价：需要保存完整梯度，内存占用高。

立即更新

每一层梯度算完就立即发起同步，与后续层的反向计算重叠。

内存占用较低，通信可被计算掩盖。

代价：通信次数多，小消息的启动开销占比高。

权衡 批量更新通信次数少但内存占用高；立即更新通信次数多但内存占用较低。选择取决于带宽与显存哪一侧更紧张。

两种梯度同步机制

机制 1

参数服务器（Parameter Server）

指定一个设备作为服务器。其他设备将计算的梯度发送给服务器；服务器聚合所有梯度，通常计算均值 $\bar{g}$，然后将聚合结果 $\bar{g}$ 广播回所有设备。

服务器收集梯度时，发送完梯度的设备可能处于空闲等待状态，降低了资源利用率。

机制 2

All-Reduce 集合通信

所有设备通过高效的集合通信操作直接交换梯度数据，最终在每个设备上得到相同的聚合梯度 $\bar{\boldsymbol{g}}$。

All-Reduce 通常是同步的：所有设备必须完成当前计算并参与通信，较快的设备要等待较慢的设备，称为“拖尾”现象。

$$\bar{\boldsymbol{g}} = \texttt{All-Reduce}\left(\boldsymbol{g}_0, \boldsymbol{g}_1, \cdots, \boldsymbol{g}_{N-1}\right) = \frac{1}{N}\sum_{i=0}^{N-1}\boldsymbol{g}_i$$

$\boldsymbol{g}_i$ 为设备 i 在本地批次上算出的梯度；全规约后每张卡都持有相同的均值梯度，从而保证参数一致。

异步数据并行

同步模式要求所有设备等待梯度全规约完成后再同步更新参数。异步数据并行则不同：各节点完成本地梯度计算后，无需等待其他节点，立即使用这些梯度更新全局模型参数。

$$\boldsymbol{W}_{t+1} = \boldsymbol{W}_t - \eta \boldsymbol{g}_A(\boldsymbol{W}_{t-\tau}), \quad \tau \geqslant 0$$

节点 A 基于版本 $\boldsymbol{W}_t$ 计算梯度时，参数服务器可能已被节点 B 更新为 $\boldsymbol{W}_{t+1}$，导致节点 A 的梯度相对于最新参数已过时。τ 表示滞后步数。

优势

没有等待

避免快速节点等待慢速节点，提高硬件利用率。

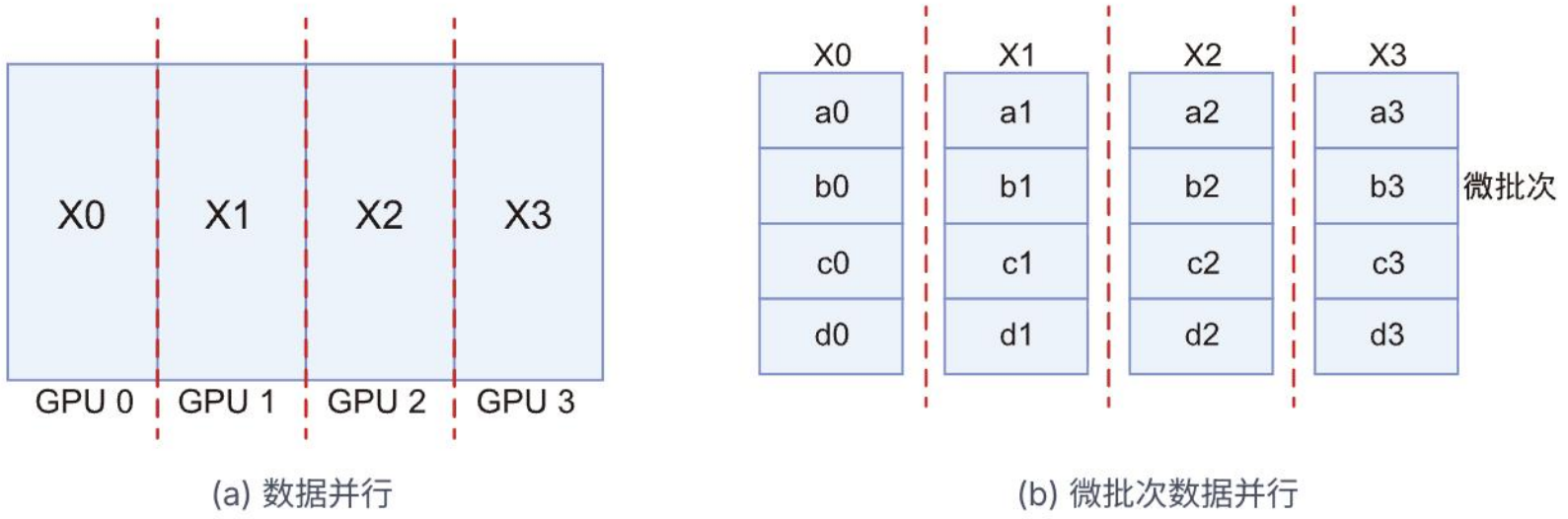
劣势

收敛风险

过时梯度可能导致训练不稳定和收敛困难；参数服务器可能成为通信瓶颈（尤其单服务器架构）。

随着 GPU 数量增加，参数版本差异加剧，可能显著影响模型收敛性。

环形全规约



(a) 数据并行　　(b) 微批次数据并行

图 9.11　环形全规约数据划分示意图

- 共有 n 个 GPU，每个 GPU j 持有全局数据的一个互斥分块 $\boldsymbol{X}_j \in \mathbb{R}^{b\times d}$，$b$ 为局部批次大小。
- 每个分块可进一步划分为 k 个微批次 $\boldsymbol{X}_{ij} \in \mathbb{R}^{b'\times d}$，其中 $b' = b/k$。
- 通信采用环形拓扑：每个 GPU i 将数据发送给后继 $(i+1)\%n$，并从前驱 $(i-1)\%n$ 接收；GPU $n-1$ 的后继是 GPU 0。

图 《大语言模型：训练、精调与对齐》图 9.11；第 9.2.1 节

环形全规约阶段一

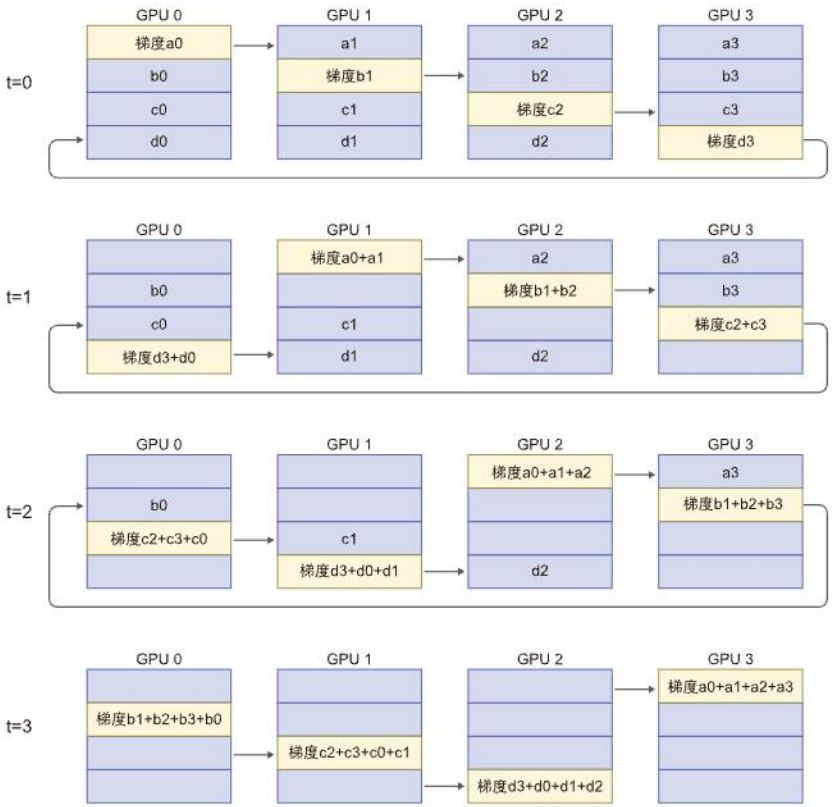


图 9.12　环形全规约：规约-散播阶段

- 目标：在 $n-1$ 步内，每个 GPU 最终持有其中一个微批次数据块梯度的总和。
- 每一步，每个 GPU i 计算当前持有数据的梯度，把梯度分块（或分块和）发送给后继 GPU。
- 同时从前驱接收一个分块，并与本地对应的分块累加。
- 经过 $n-1$ 步后，GPU i 存储了全局梯度总和的一个完整分块 $\tilde{G}_i$。

$$\tilde{G}_i = \sum_{j=1}^{n} G_{i,j}$$

$\tilde{G}_i$ 为第 i 个梯度分块在所有 GPU 上的总和，最终只落在 GPU i 上；其余分块的求和结果落在其余 GPU 上。

图 《大语言模型：训练、精调与对齐》图 9.12；第 9.2.1 节

环形全规约阶段二

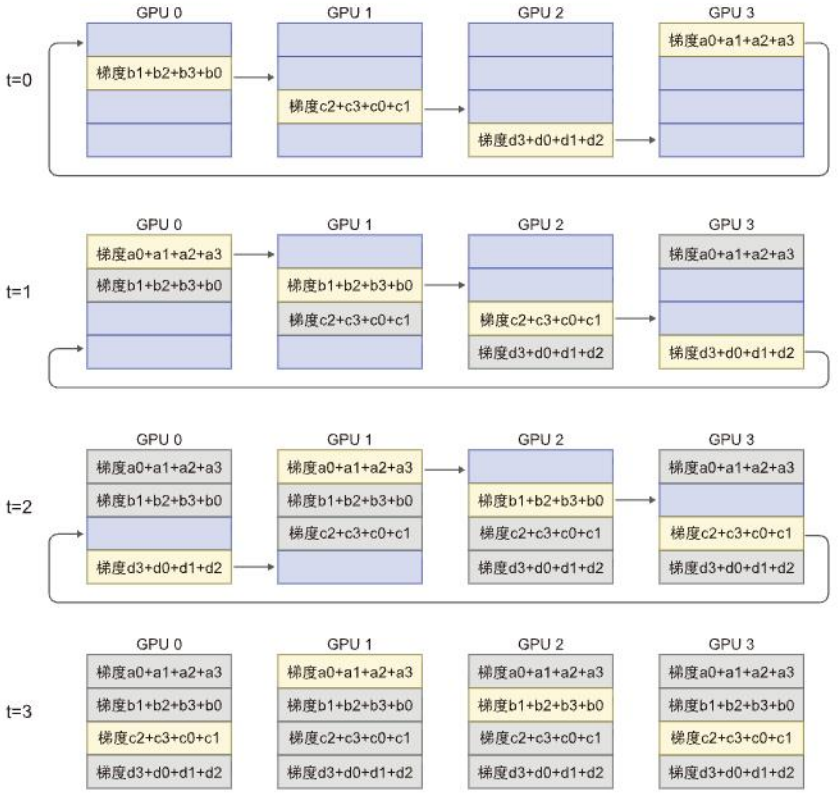


图 9.13　环形全规约：全收集阶段

- 目标：在 $n-1$ 步内，把规约-散播阶段产生的 n 个分块总和广播到所有 GPU。
- 每一步，每个 GPU i 将当前持有的分块发送给后继，同时从前驱接收一个尚未持有的分块。
- 经过 $n-1$ 步，每个 GPU 都存储了所有 n 个分块，即完整的全局梯度总和，实现了梯度一致性。

两个阶段合计 $2(n-1)$ 步，每步传输 $O(N/n)$ 数据：总传输量 $O(N)$，且所有链路并行工作。

$$\boldsymbol{G}_{\text{total}} = \sum_{i=0}^{n-1} \tilde{G}_i = \sum_{j=0}^{n-1} \boldsymbol{g}_j$$

全收集结束后，每张卡上的 $\boldsymbol{G}_{\text{total}}$ 相同，正是各设备梯度的总和（按需再除以 n 得到均值）。

图 《大语言模型：训练、精调与对齐》图 9.13；第 9.2.1 节

4 个 GPU 的环形全规约

假设全局批次被划分为 4 个互斥分块 $\boldsymbol{X}_0, \boldsymbol{X}_1, \boldsymbol{X}_2, \boldsymbol{X}_3$，分别由 GPU0–GPU3 持有。每个 GPU 在其微批次分块（如 $\boldsymbol{X}_0$ 分为 a_0, b_0, c_0, d_0）上计算梯度，一步操作后各 GPU 得到对应的 $G_{a0}, G_{b1}, G_{c2}, G_{d3}$。

初始化

4 块互斥数据

a_0, b_0, c_0, d_0

→

规约-散播

3 步

各得一块总和

→

全收集

3 步

补齐全部块

阶段	步数	阶段结束时的状态
规约-散播	4 − 1 = 3	GPU0 持有 G_b 的总和，GPU1 持有 G_c 的总和，GPU2 持有 G_d 的总和，GPU3 持有 G_a 的总和
全收集	3	每张卡都获得 $G_{\text{total}} = G_a + G_b + G_c + G_d$ 的所有分块

分块与持有者的错位（GPU0 拿 G_b 而非 G_a）来自环形传递的步进：每步发送后，当前块归属于后继链路上的位置。

环形与树形的对比

全规约的实现主要有两种形式：环形全规约与树形全规约。两者总数据传输量均为 $O(N)$（N 为梯度总量），但通信模式和时间复杂度存在显著差异。

环形全规约

每个节点只与相邻节点通信。通信步数 $O(n)$，每步传输 $O(N/n)$。

优势：所有链路并行传输，带宽利用率高，适合大规模集群。缺点：延迟随节点数线性增长。

树形全规约

采用树状拓扑（如二叉树），通信步数 $O(\log n)$，时间复杂度更低。

缺点：高层级节点（尤其根节点）的链路带宽压力集中，容易成为瓶颈并引发网络拥塞。

形式	通信步数	单步数据量	瓶颈位置
环形	$O(n)$	$O(N/n)$	延迟随节点数增长
树形	$O(\log n)$	逐层变化	根节点与高层链路的带宽

因此，尽管树形全规约的步数复杂度更优，但在大规模节点场景下，环形全规约凭借均衡的带宽利用率和可扩展性，往往能获得更优的实际吞吐量。

依据 《大语言模型：训练、精调与对齐》第 9.2.1 节

张量并行

当模型参数规模过大，导致单个 GPU 的显存无法容纳所有参数时，或者仅为了分解计算规模，需要将模型参数切分并部署到多个 GPU 上存储。张量并行的核心思想，是将神经网络层中的权重矩阵进行切分，使每个设备仅需存储部分权重。

收益

显存按比例下降

每个设备只存部分权重，相应地，所需存储的激活值和优化器状态量也随之减少。

代价

通信随粒度上升

随着切分粒度增加（使用更多设备并行），设备间的通信开销也会显著增大。

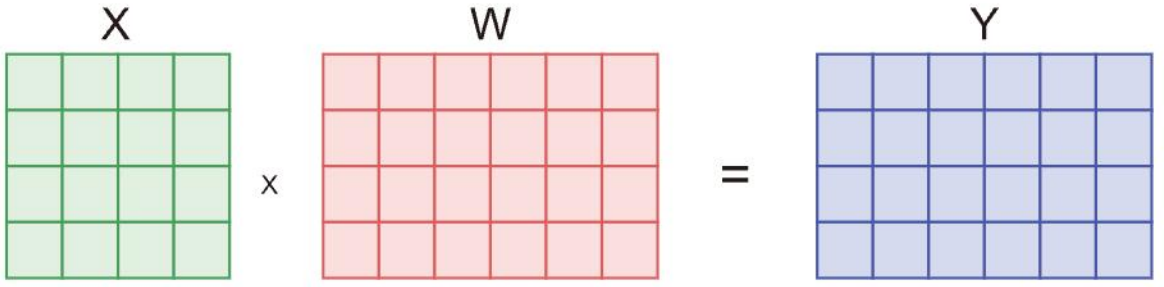
本质

用通信换显存

实际加速效果受限于设备间的通信带宽和延迟，是以增加通信开销换取降低单设备显存的优化策略。

下面先分析矩阵乘法中的不同行列切分策略，进而探讨 Transformer 架构下的张量并行设计。

矩阵乘法的切分



标准矩阵计算

$$\boldsymbol{Y} = \boldsymbol{X}\boldsymbol{W}, \quad \boldsymbol{Y} \in \mathbb{R}^{2N \times 2R} \tag{9-1}$$

在张量并行组内使用 n 个设备，为简化说明以 $n = 2$ 为例。给定权重矩阵 $\boldsymbol{W} \in \mathbb{R}^{2D \times 2R}$ 和输入矩阵 $\boldsymbol{X} \in \mathbb{R}^{2N \times 2D}$。

图 9.14　标准矩阵乘法示意图

当 $\boldsymbol{W}$ 超出单个设备的显存容量时，有两种自然的切法：按列切成左右两块，或按行切成上下两块。两种切法在前向与反向中需要的通信位置恰好相反。

列切分

$\boldsymbol{W} = [\boldsymbol{A}_1, \boldsymbol{A}_2]$，每卡持有一部分**输出列**；前向各算一半、反向需规约输入梯度。

行切分

$\boldsymbol{W} = [\boldsymbol{B}_1; \boldsymbol{B}_2]$，每卡持有一部分**输入行**；前向需规约输出、反向各算一半。

图 《大语言模型：训练、精调与对齐》图 9.14；式 (9-1)

列切分

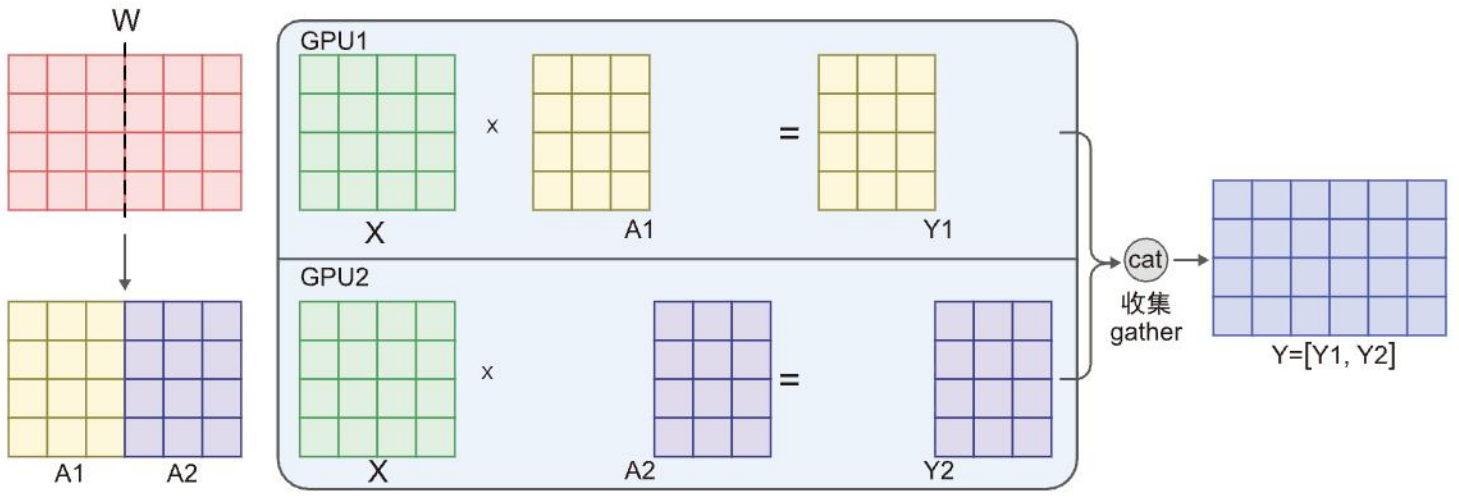


图 9.15　权重矩阵列切分的张量并行示意图

$$[\boldsymbol{A}_1, \boldsymbol{A}_2] = \texttt{Split_col}(\boldsymbol{W}) \tag{9-2}$$

$$\boldsymbol{Y} = \boldsymbol{X}[\boldsymbol{A}_1, \boldsymbol{A}_2] = [\boldsymbol{Y}_1, \boldsymbol{Y}_2] \tag{9-3}$$

$$\boldsymbol{Y}_1 = \boldsymbol{X}\boldsymbol{A}_1, \qquad \boldsymbol{Y}_2 = \boldsymbol{X}\boldsymbol{A}_2$$

$\boldsymbol{A}_1, \boldsymbol{A}_2 \in \mathbb{R}^{2D\times R}$，分别存储于 GPU1 和 GPU2；$\boldsymbol{Y}_1, \boldsymbol{Y}_2 \in \mathbb{R}^{2N\times R}$。

- 每块 GPU 需要完整的输入 $\boldsymbol{X}$，因此需先将 $\boldsymbol{X}$ 广播至所有设备。
- 最终通过收集操作得到完整输出 $\boldsymbol{Y} \in \mathbb{R}^{2N\times 2R}$。

列切分没有把输入切开，所以每张卡都要读整份 $\boldsymbol{X}$；它的通信集中在前向的广播与反向的规约。

图 《大语言模型：训练、精调与对齐》图 9.15；式 (9-2)、(9-3)

列切分的反向传播

$$\mathrm{d}\boldsymbol{A}_1 = \boldsymbol{X}^T \mathrm{d}\boldsymbol{Y}_1, \quad \mathrm{d}\boldsymbol{A}_2 = \boldsymbol{X}^T \mathrm{d}\boldsymbol{Y}_2 \tag{9-4}$$

$$\mathrm{d}\boldsymbol{X} = \mathrm{d}\boldsymbol{Y}_1 \boldsymbol{A}_1^T + \mathrm{d}\boldsymbol{Y}_2 \boldsymbol{A}_2^T \tag{9-5}$$

$$= \texttt{All-Reduce}\left(\mathrm{d}\boldsymbol{Y}_1 \boldsymbol{A}_1^T,\ \mathrm{d}\boldsymbol{Y}_2 \boldsymbol{A}_2^T\right) \tag{9-6}$$

给定损失对输出的梯度 $\mathrm{d}\boldsymbol{Y} \in \mathbb{R}^{2N\times 2R}$，按列切分为 $\mathrm{d}\boldsymbol{Y}_1, \mathrm{d}\boldsymbol{Y}_2 \in \mathbb{R}^{2N\times R}$。

第 1 步 · 权重梯度

本地独立计算

每张卡用自己那份 $\mathrm{d}\boldsymbol{Y}_i$ 与本地的 $\boldsymbol{A}_i$、完整的 $\boldsymbol{X}^T$ 相乘，得到 $\mathrm{d}\boldsymbol{A}_1, \mathrm{d}\boldsymbol{A}_2 \in \mathbb{R}^{2D\times R}$，无需通信。

第 2 步 · 输入梯度

跨设备全规约

完整输入梯度 $\mathrm{d}\boldsymbol{X} \in \mathbb{R}^{2N\times 2D}$ 是两份局部贡献之和，通过 All-Reduce 求和得到。

式 (9-5) 拆成两项正是因为输出被按列切开：每张卡只掌握一部分 $\mathrm{d}\boldsymbol{Y}$，对应的输入梯度也只是部分和。

行切分

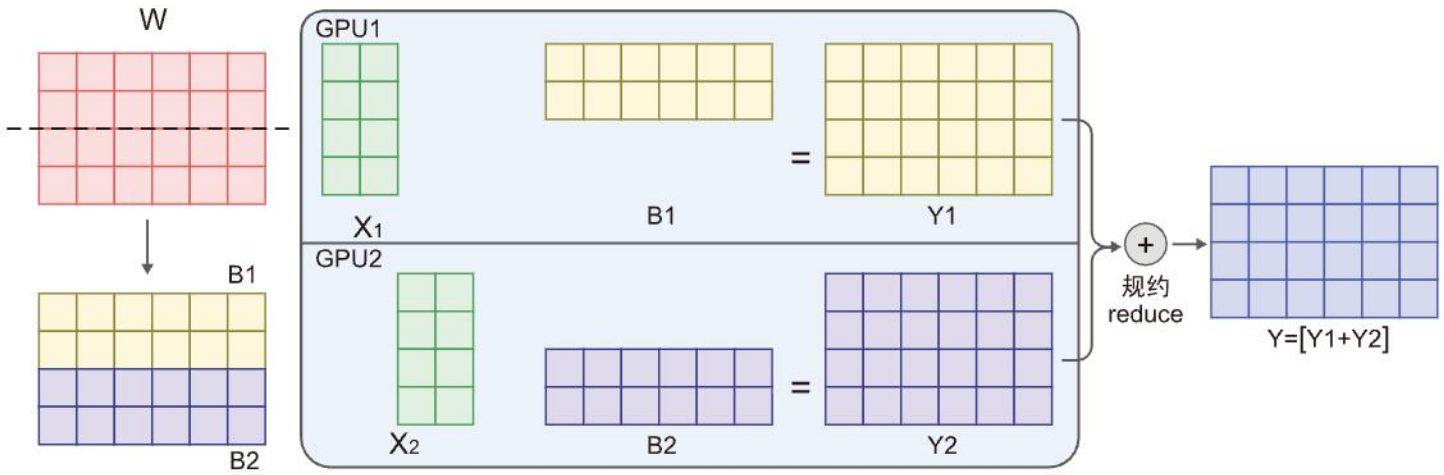


图 9.16　权重矩阵行切分的张量并行示意图

$$\begin{bmatrix} \boldsymbol{B}_1 \\ \boldsymbol{B}_2 \end{bmatrix} \leftarrow \texttt{Split_row}(\boldsymbol{W}) \quad (9-$$

$$[\boldsymbol{X}_1, \boldsymbol{X}_2] \leftarrow \texttt{Split_col}(\boldsymbol{X}) \ (9-$$

$$\boldsymbol{Y} = [\boldsymbol{X}_1, \boldsymbol{X}_2] \begin{bmatrix} \boldsymbol{B}_1 \\ \boldsymbol{B}_2 \end{bmatrix} = \underbrace{\boldsymbol{X}_1\boldsymbol{B}_1}_{\boldsymbol{Y}_1} + \underbrace{\boldsymbol{X}_2\boldsymbol{B}_2}_{\boldsymbol{Y}_2}$$

$\boldsymbol{B}_1 \in \mathbb{R}^{D\times 2R}$ 存于 GPU1，$\boldsymbol{B}_2 \in \mathbb{R}^{D\times 2R}$ 存于 GPU2，$\boldsymbol{X}_1, \boldsymbol{X}_2 \in \mathbb{R}^{2N\times D}$。

- $\boldsymbol{Y}_1, \boldsymbol{Y}_2 \in \mathbb{R}^{2N\times 2R}$ 分别位于两张卡，需通过 $\texttt{All-Reduce}(\cdot)$ 求和得到完整输出 $\boldsymbol{Y} \leftarrow \boldsymbol{Y}_1 + \boldsymbol{Y}_2$。
- 行切分把输入也按列切开了，因此每张卡只需读一半 $\boldsymbol{X}$，代价是前向必须做一次规约。

图 《大语言模型：训练、精调与对齐》图 9.16；式 (9-7)–(9-9)

行切分的反向传播

$$\mathrm{d}\boldsymbol{B}_1 = \boldsymbol{X}_1^T \mathrm{d}\boldsymbol{Y}, \quad \mathrm{d}\boldsymbol{B}_2 = \boldsymbol{X}_2^T \mathrm{d}\boldsymbol{Y} \tag{9-10}$$

$$\mathrm{d}\boldsymbol{X}_1 = \mathrm{d}\boldsymbol{Y} \boldsymbol{B}_1^T, \quad \mathrm{d}\boldsymbol{X}_2 = \mathrm{d}\boldsymbol{Y} \boldsymbol{B}_2^T \tag{9-11}$$

$$\mathrm{d}\boldsymbol{X} \leftarrow [\mathrm{d}\boldsymbol{X}_1, \mathrm{d}\boldsymbol{X}_2] \tag{9-12}$$

$\mathrm{d}\boldsymbol{Y} \in \mathbb{R}^{2N \times 2R}$ 需先广播至所有设备；$\mathrm{d}\boldsymbol{B}_1, \mathrm{d}\boldsymbol{B}_2 \in \mathbb{R}^{D \times 2R}$，$\mathrm{d}\boldsymbol{X}_1, \mathrm{d}\boldsymbol{X}_2 \in \mathbb{R}^{2N \times D}$。

广播

$\mathrm{d}\boldsymbol{Y}$ 发到所有卡

→

权重梯度

各卡独立算 $\mathrm{d}\boldsymbol{B}_i$ (9-10)

→

输入梯度

各卡独立算 $\mathrm{d}\boldsymbol{X}_i$ (9-11)

→

拼接

`Gather` 得完整 $\mathrm{d}\boldsymbol{X}$ (9-12)

与列并行对照：列并行的通信在反向的输入梯度上，行并行的通信在前向的输出求和与反向的输出梯度广播上。这一“互补”关系正是 FFN 与注意力层能各取所长的基础。

依据 《大语言模型：训练、精调与对齐》第 9.2.2 节（式 (9-10)–(9-12)）

FFN 层的张量并行

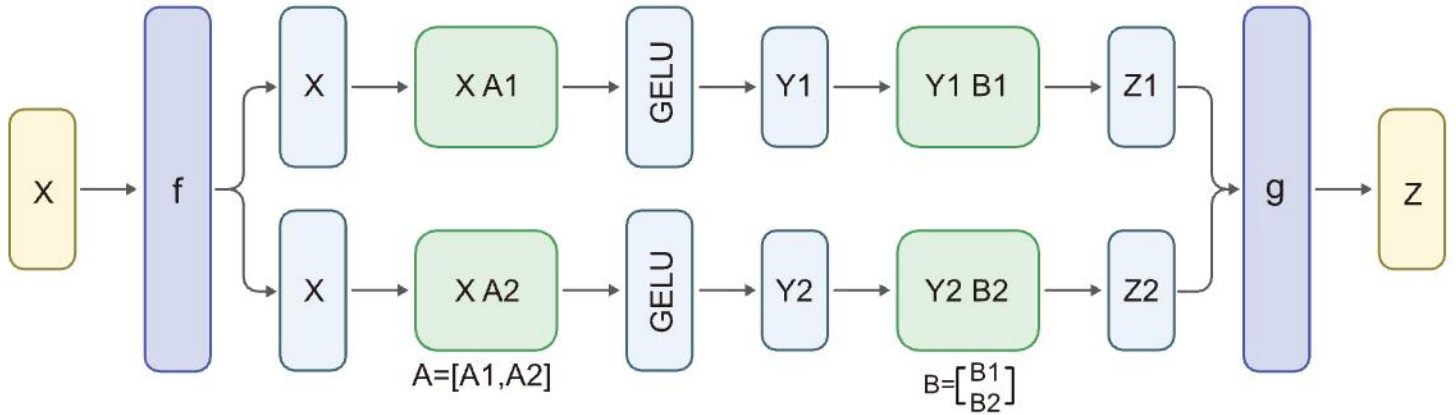


图 9.17　MLP 层的张量并行切分方式

$$\boldsymbol{Y} = \sigma(\boldsymbol{X}\boldsymbol{W}_1 + \boldsymbol{B}_1)\boldsymbol{W}_2 + \boldsymbol{B}_2 \tag{9-13}$$

$$[\boldsymbol{Y}_1, \boldsymbol{Y}_2] = [\mathrm{GELU}(\boldsymbol{X}\boldsymbol{A}_1), \mathrm{GELU}(\boldsymbol{X}\boldsymbol{A}_2)] \tag{9-14}$$

$$\boldsymbol{Z} = [\boldsymbol{Y}_1, \boldsymbol{Y}_2]\begin{bmatrix}\boldsymbol{B}_1 \\ \boldsymbol{B}_2\end{bmatrix} = \boldsymbol{Y}_1\boldsymbol{B}_1 + \boldsymbol{Y}_2\boldsymbol{B}_2 \tag{9-15}$$

$\boldsymbol{W}_1 \in \mathbb{R}^{D\times 4D}$ 列切分为 $\boldsymbol{A}_1, \boldsymbol{A}_2 \in \mathbb{R}^{D\times 2D}$；$\boldsymbol{W}_2 \in \mathbb{R}^{4D\times D}$ 行切分为 $\boldsymbol{B}_1, \boldsymbol{B}_2 \in \mathbb{R}^{2D\times D}$。

为什么必须这样配？若对 $\boldsymbol{W}_1$ 采用行切分，由于激活函数的非线性特性，先求和再激活与先激活再求和并不相等：

$$\mathrm{GELU}(\boldsymbol{X}_1\boldsymbol{W}_{11} + \boldsymbol{X}_2\boldsymbol{W}_{12}) \neq \mathrm{GELU}(\boldsymbol{X}_1\boldsymbol{W}_{11}) + \mathrm{GELU}(\boldsymbol{X}_2\boldsymbol{W}_{12}) \tag{9-14}$$

FFN 张量并行的推导

$$\boldsymbol{Z} = \boldsymbol{Y}_1\boldsymbol{B}_1 + \boldsymbol{Y}_2\boldsymbol{B}_2 = \boldsymbol{Z}_1 + \boldsymbol{Z}_2 \tag{9-17}$$

$$\boldsymbol{Z} = \texttt{All-Reduce}(\boldsymbol{Z}_1, \boldsymbol{Z}_2) \tag{9-18}$$

$\boldsymbol{Y}_1, \boldsymbol{Y}_2 \in \mathbb{R}^{2N\times 2D}$，$\boldsymbol{Z}, \boldsymbol{Z}_1, \boldsymbol{Z}_2 \in \mathbb{R}^{2N\times D}$。式 (9-17) 是局部矩阵乘，式 (9-18) 是前向必须的一次规约。

通信账

一次前向 + 一次反向

前向需要一次 All-Reduce 求出 $\boldsymbol{Z}$；反向传播中，计算输入梯度 $\mathbf{d}\boldsymbol{X}$ 时需再次执行 All-Reduce。

存储账

激活值冗余

在计算梯度 $\mathbf{d}\boldsymbol{A}_1, \mathbf{d}\boldsymbol{A}_2$ 时，各 GPU 前向可能存储了 $\boldsymbol{X}$ 的值用于反向计算，会导致一定的数据占用冗余。

这一对“列切分 + 行切分”的组合是 Megatron-LM 张量并行的核心模式：在层内部完成切分与计算，只在层的出口处做一次规约，把通信压到最少。

注意力层的张量并行

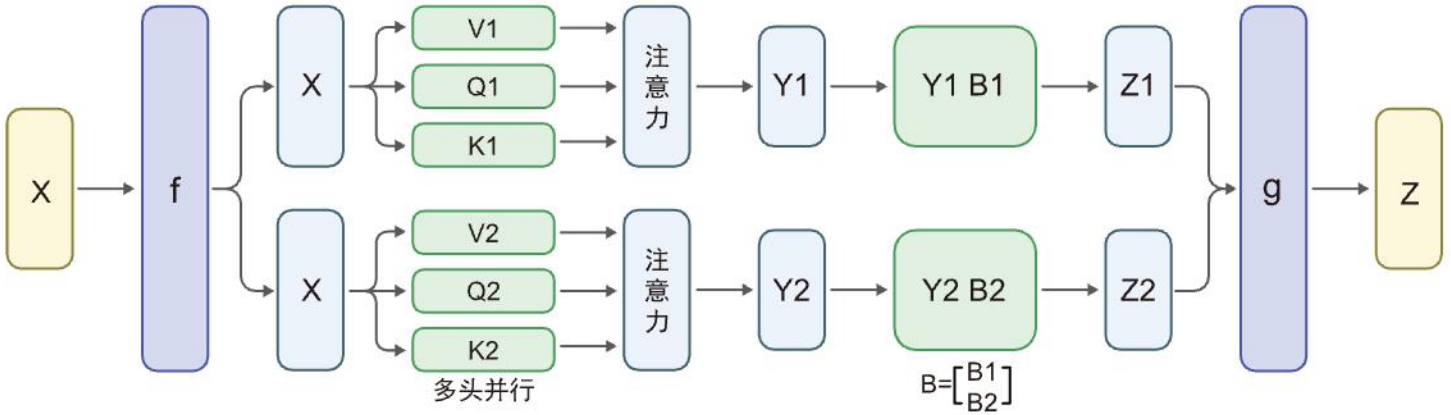


图 9.18　自注意力层头并行策略

$$\boldsymbol{W}_q^{(i)}, \boldsymbol{W}_k^{(i)}, \boldsymbol{W}_v^{(i)} \in \mathbb{R}^{d \times d'} \qquad (9\text{-}1$$

$$\boldsymbol{Z}^{(i)} = \texttt{Attn}\left(\boldsymbol{X}\boldsymbol{W}_q^{(i)}, \boldsymbol{X}\boldsymbol{W}_k^{(i)}, \boldsymbol{X}\boldsymbol{W}_v^{(i)}\right)$$

$d' = d/N$，N 为设备数，设注意力头数与设备数相同；$\boldsymbol{X}$ 为完整输入。

- $\boldsymbol{W}_q, \boldsymbol{W}_k, \boldsymbol{W}_v$ 按列切分，每个设备处理部分注意力头。
- 输出投影矩阵 $\boldsymbol{W}_o$ 按行切分，最终 All-Reduce 规约结果。

这一方案称为头并行。它避免了先计算完整 $\backslash\mathrm{matQKV}$ 矩阵再切分散播的通信开销；对于分组查询注意力，若使用键值头共享，则需在组内同步 $\boldsymbol{W}_k^{(i)}, \boldsymbol{W}_v^{(i)}$ 参数。

其他层的张量并行

归一化层

参数量小，两种都行

可以在每个设备上冗余存储，也可以沿特征维度切分参数。

输入来自列并行层时，求均值与方差需设备间同步；输入来自行并行层时，整个归一化可在各设备内独立完成。

词嵌入层

词表大，必须切

（1）**词表并行**：将词表划分为若干子集分配给不同设备，每个设备存储完整特征维度的部分词向量。

（2）**特征维度并行**：将词向量沿特征维度切分，各设备存储所有词元的部分特征。

语言输出层

计算量大，切法有别

同样可按词表维度或特征维度划分，但投影矩阵 $\boldsymbol{W}_{\text{output}} \in \mathbb{R}^{d \times |\mathcal{V}|}$ 在前后向均涉及大规模矩阵计算，且需接 Softmax。

特征维度划分较易实现；词表划分更复杂，需特殊处理如并行交叉熵损失计算。

规模感 当 $d = 4096$、$|\mathcal{V}| = 128\,000$ 时，输出层单个权重矩阵就有约 5.2 亿个元素——它本身就是一块需要被切分的显存。

词嵌入层通过词元索引获取向量，而输出层要对所有词元做完整的矩阵乘法，这就是两者切分策略不能简单照搬的原因。

张量并行小结

张量并行的核心特征是计算过程中需频繁通信：前向传播中，行并行的中间结果需强制规约求和；反向传播中，列并行的输入梯度需强制规约求和。

对比维度	数据并行	张量并行
通信时机	仅反向传播结束后同步梯度	前向与反向计算过程中均插入通信
通信频率	每步一次（或每层一次）	每层多次，随切分粒度上升
切分对象	数据样本	层内参数矩阵
典型部署	跨节点	节点内（NVLink / NVSwitch）

设计原则

四条约束

（1）相邻层间的切分衔接；（2）前向与反向传播的计算特性；（3）通信开销与显存占用的平衡；（4）层内的计算特性。

应用

推理与训练的差别

推理场景中，张量并行因能降低单设备显存需求、提高并行计算效率被广泛采用；但频繁通信在训练或推理系统中都可能成为效率瓶颈。

流水线并行

除了张量并行，还可以在完整模型层面上将其切分成多个阶段，这种切分方法称为流水线并行。与张量并行单个 GPU 只持有参数矩阵的一部分不同，流水线并行中每个 GPU 持有其对应阶段完整的参数矩阵。

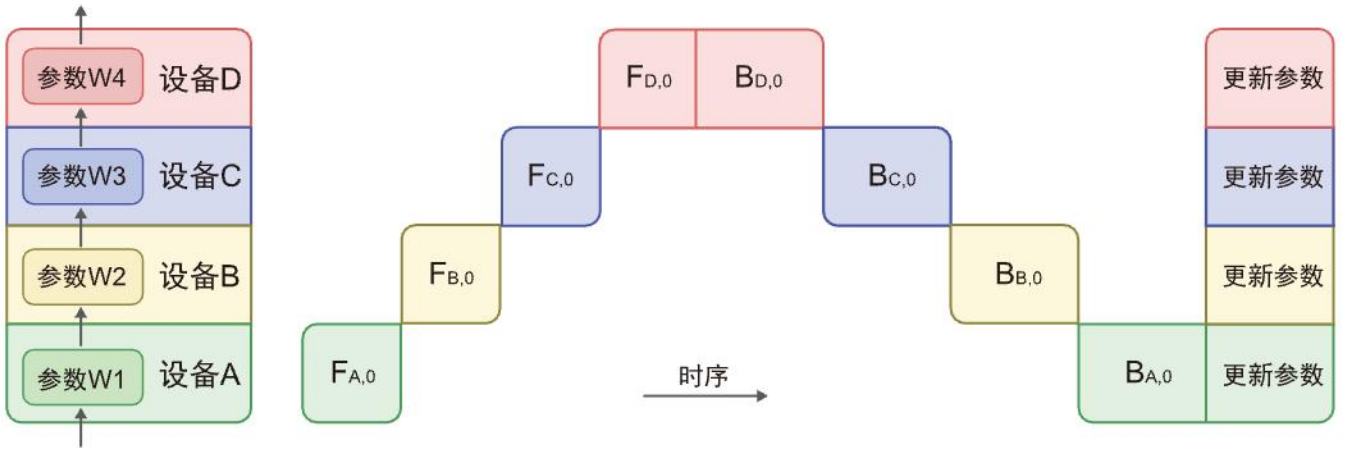


图 9.19　流水线并行中的气泡现象及其对集群利用率的影响

示例

8 层模型，4 个阶段

每两层划分为一个阶段，4 个阶段分配到 4 个 GPU 上。每个 GPU 只需存储和计算其分配阶段所需的参数和激活值。

这显著降低了单个 GPU 的显存需求，使得可以用更多 GPU 来训练更深的模型。

GPipe 提出的流水线并行方法，能够有效减少训练时单卡的显存占用。代价是设备之间必须串行传递激活值与梯度，从而产生等待。

流水线的气泡

在这种训练模式下，设备间存在等待时间。例如设备 A 是第一个完成前向计算的，但却是最后一个进行反向梯度计算的，这导致它在前向结束后需要等待很长时间才能开始反向计算。

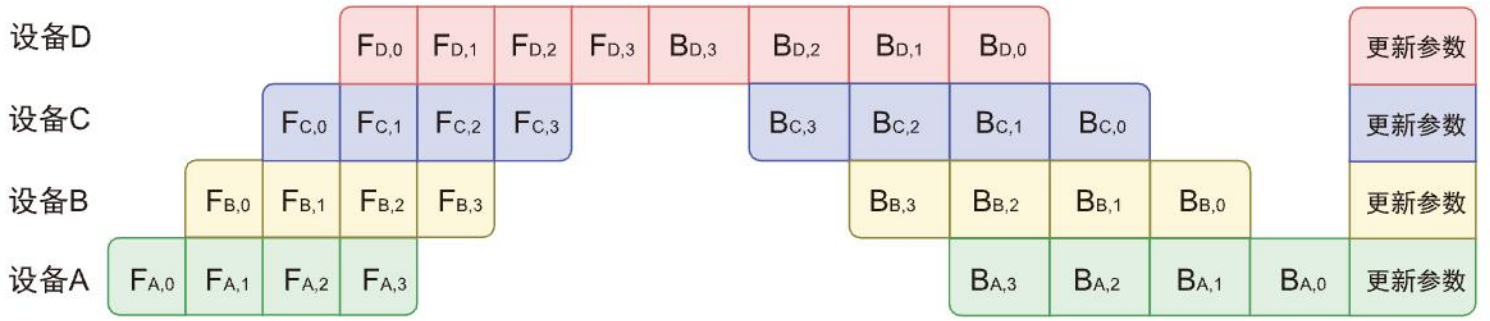
> **定义** 多 GPU 系统中设备因等待通信而处于空闲状态的时间称为气泡。流水线并行系统中气泡越少，则系统的整体计算利用率越高。

$$\text{气泡时间占比} = \frac{\text{所有 GPU 的空闲等待时间之和}}{\text{整个流水线过程的总运行时间}}$$

这是衡量气泡大小的指标：分母包含所有阶段的计算与通信，分子只统计空转。

沿用图 9.19 的记号：P 个阶段、每个批量切成 M 个微批量时，气泡占比约为 $(P-1)/(P-1+M)$。阶段数越多、微批量越少，空转越显著——这正是 GPipe 引入微批量、PipeDream 改用异步调度的原因。

GPipe



核心

批量再细分

GPipe 提出微批量的概念来配合编排流水线调度执行任务：将一个批量数据细分为多个小的计算单元。

图 9.20　微批量处理通过流水线调度减少气泡率

前向调度

① 第 1 个 GPU 先算 $\mathrm{F}_{A,0}$；② 把结果传给第 2 个 GPU；③ 下一时间步，第 1 个 GPU 算 $\mathrm{F}_{A,1}$，同时第 2 个 GPU 处理 $\mathrm{F}_{B,0}$；④ 持续进行直至所有微批量完成前向。

反向调度

① 第 4 个 GPU 先算 $\mathrm{B}_{D,3}$；② 把梯度传给第 3 个 GPU；③ 下一时间步，第 4 个 GPU 算 $\mathrm{B}_{D,2}$，第 3 个 GPU 处理 $\mathrm{B}_{C,3}$；④ 全部完成后各阶段同步更新参数。

图 《大语言模型：训练、精调与对齐》图 9.20；第 9.2.3 节

微批量的代价

微批量处理的核心价值在于减少气泡时间占比：通过精细的流水线调度，使更多设备保持并行计算状态。理论上，微批量数量越多，气泡占比越低，系统效率越高。

收益：气泡变小

更多设备在同一时刻有活可干，硬件利用率上升。

气泡占比大致随微批量数量的增加而下降。

代价：激活值变多

增加微批量数量会大幅增加激活值的存储需求，显存压力随之上升。

在显存与气泡之间需要折中，而不是一味加大微批量数。

缓解 采用重计算策略：前向传播中丢弃部分激活值，反向传播时按需重新计算。该技术称为梯度检查点。

简单激活

对于 ReLU 等计算简单的激活函数，可仅保留激活前的值，重计算开销较小，但可减少约 50% 的显存占用。

复杂层

计算复杂的层可保留激活值，避免重计算开销，把省显存的任务交给代价更低的位置。

PipeDream

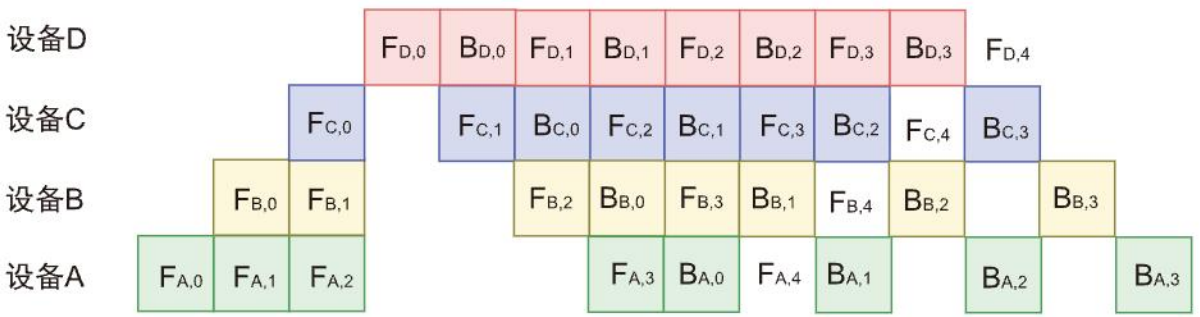


图 9.21　PipeDream 的异步流水线并行策略

- GPipe 中所有设备同步进行参数更新；PipeDream 采用异步更新策略，允许不同设备在不同时间更新参数。
- 通过在单个 GPU 内交错执行前向传播和反向传播计算（通常称这种复合操作为 1F1B），最大限度减少硬件空闲时间。
- 完成微批量计算后立即释放激活值，显著降低显存需求。

阶段 1

预热

逐步填充流水线，主要做前向。

阶段 2

稳定

流水线充满，设备同时在算。

异步流水线的问题

当设备 A 处理第 5 个微批量数据（$\mathrm{F}_{A,4}$）时，各设备上的模型参数已基于不同数据批次进行了更新。

现象

版本分叉

设备 A 的参数仅使用数据 1 的梯度更新过，而设备 D 的参数已使用数据 1–4 的梯度进行了四次更新。同一时刻不同阶段的参数版本不一致。

对策

参数快照

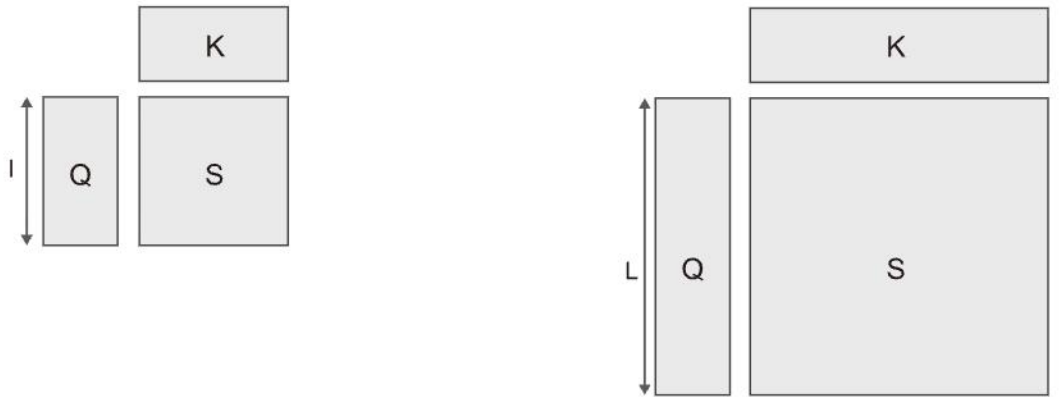
为确保梯度计算正确性，PipeDream 需要为每个微批量存储对应的参数快照，这会显著增加存储开销。

主流 1F1B 实现会采用梯度累计技术：并非计算完梯度就更新参数，而是达到更新条件再进行参数同步，保证计算时前向与反向的参数版本唯一。

后续研究进一步优化了流水线并行：Chimera 提出双流水线并行方法；ZeroBubble 提出将参数更新与输入梯度传递操作异步并行执行，该复合操作被称为 1F1B1W；DeepSeek-V3 提出的 DualPipe 通过重叠计算与通信操作，有效隐藏了专家并行中全交换通信的时间开销。

从 1F1B 到 1F1B1W 再到 DualPipe，改进的方向始终一致：把更多的操作塞进同一段时间里，让等待消失。

长序列的计算代价



(a) 注意力分数 1　　(b) 注意力分数 2

图 9.22(a)(b)　块注意力计算示意图：序列变长，分数矩阵迅速变大

长篇小说、工程代码等语言数据包含较长的文本。随着序列长度 L 的线性增长，注意力计算的时间和空间复杂度呈平方级增长：

$$\mathcal{O}(L^2) \text{ 的时间与空间} \Longrightarrow \text{单 GPU 不再可行}$$

这使得在单个 GPU 上处理更长序列的注意力计算变得不可行。

上下文并行

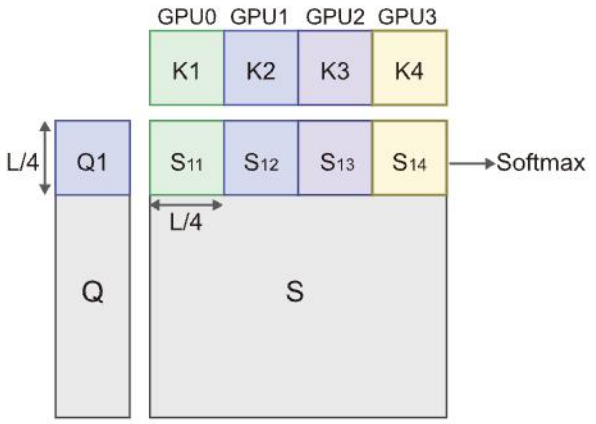


图 9.22(c)　分块注意力分数

一种解决方案是在序列长度维度上切分数据，称为序列并行或上下文并行：每个 GPU 仅接收并存储一个子序列片段。

本地计算

局部注意力

每个 GPU 只能基于其本地片段进行局部注意力计算，注意力分数矩阵被切成若干块。

核心挑战

如何规约

如何高效地将分布在多个 GPU 上的局部注意力结果规约起来，形成最终的全局注意力输出。

前馈层的计算在序列维度上完全独立，不会引入额外通信开销。

上下文并行的难点

在计算注意力前，将词向量投影成查询矩阵 $\boldsymbol{Q}$、键矩阵 $\boldsymbol{K}$ 和值矩阵 $\boldsymbol{V}$。在上下文并行方案中，将 $\boldsymbol{Q}$、$\boldsymbol{K}$、$\boldsymbol{V}$ 沿序列维度切分，并把不同的 KV 块分配到不同的 GPU 上。

$$P_{ij} = \frac{\exp(S_{ij})}{\sum_{k=1}^{L} \exp(S_{ik})}$$

由于 Softmax 函数的非线性特性，计算全局归一化的注意力分数面临挑战：需要规约所有 GPU 上的局部注意力分数信息来计算全局 Softmax 分母项，再将归一化因子分发给各 GPU。

本地分数

各卡算本地 S_{ij}

→

规约分母

跨卡收集 $\exp(S_{ik})$ 的和

→

分发因子

把全局分母发回各卡

→

本地输出

归一化后加权求和

这个过程会引入显著的通信开销——每一步都要等所有卡算完才能继续，正好与流水线中的气泡同源。

环形注意力

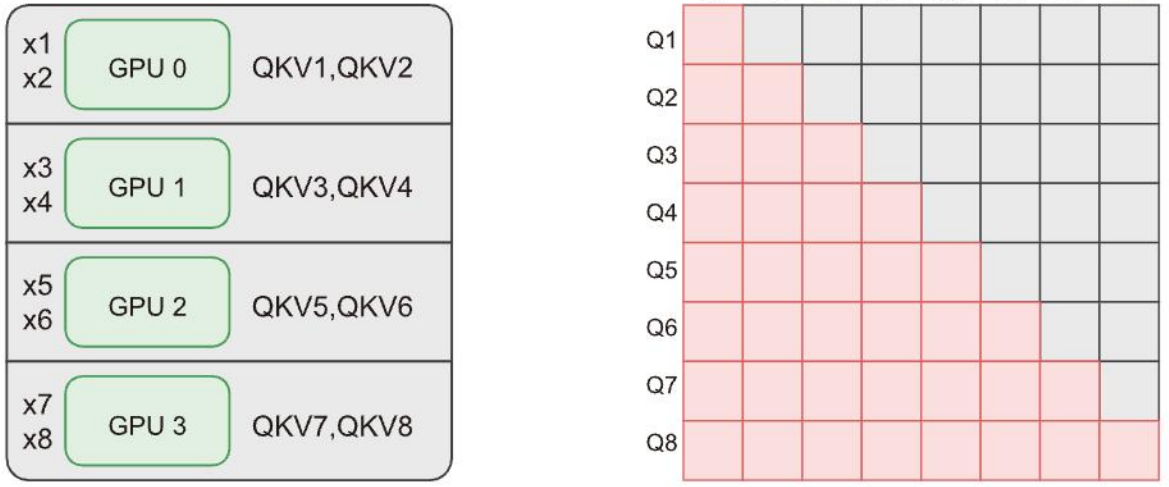


图 9.23　KV 序列切分

借助 Flash Attention-V2 的分块注意力计算思想，采用环形注意力算法实现分布式环境的分块注意力计算。初始时，每个 GPU 仅有本地的 KV 块。

- 每个 GPU 加载一个固定的 $\boldsymbol{Q}$ 块。
- 算法通过环形通信拓扑，在多个步骤间依次传递 $\boldsymbol{K}$、$\boldsymbol{V}$ 块。
- 每个步骤中，GPU 利用当前可用的 KV 块计算部分注意力结果，并逐步累积更新最终的注意力输出 $\boldsymbol{O}$。

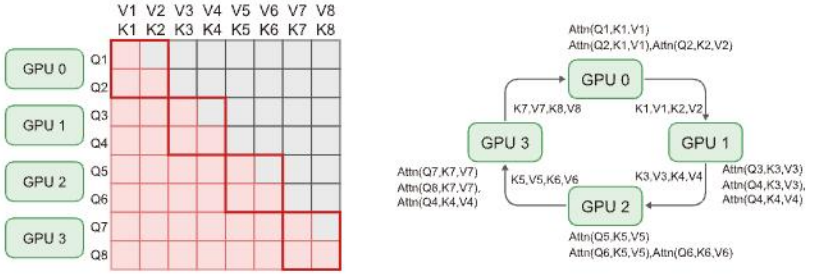
$$N \text{ 个 GPU：} \quad N-1 \text{ 次环形传递} + N \text{ 次计算} \quad \Longrightarrow \quad \text{每卡得到完整输出}$$

环状注意力在数学上与标准的注意力计算等价。

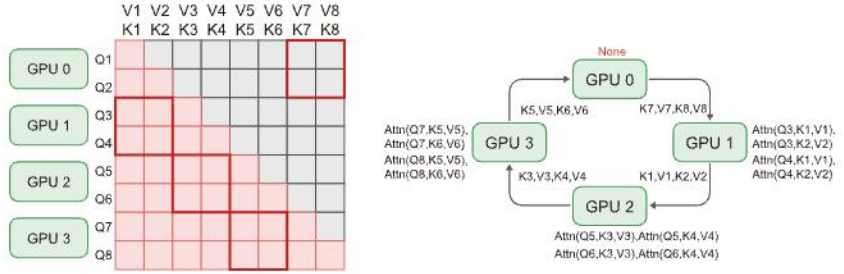
图 《大语言模型：训练、精调与对齐》图 9.23；第 9.2.4 节

环形注意力的计算过程

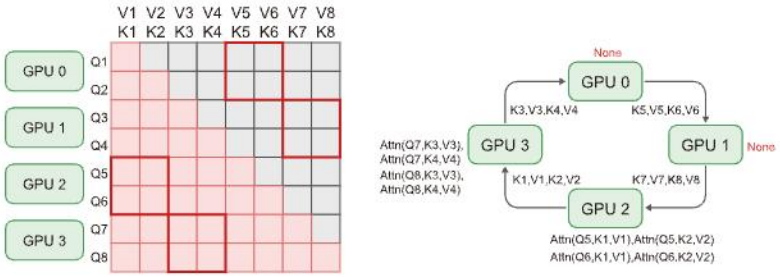
$N = 4$ 时共 4 个计算步骤：每个 GPU 持有固定的 $\boldsymbol{Q}_i$，每一轮把 $\boldsymbol{K}, \boldsymbol{V}$ 环传给后继，同时用收到的块累加本地的部分输出与归一化因子。第 3 步结束时，$\boldsymbol{Q}_i$ 已看过全部 KV，累积结果就是完整的 $\boldsymbol{O}_i$。



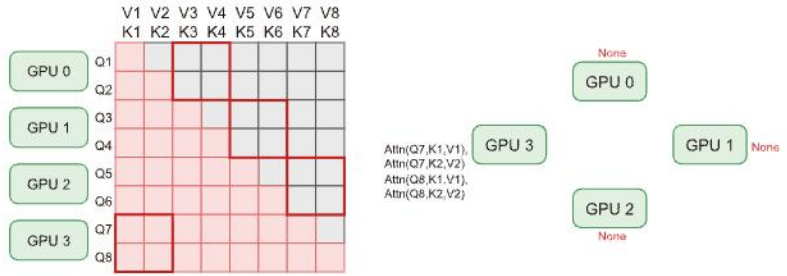
(a) 第 0 步　各卡只用本地 $\boldsymbol{K}_i, \boldsymbol{V}_i$



(b) 第 1 步　传递一轮后累加前驱的块



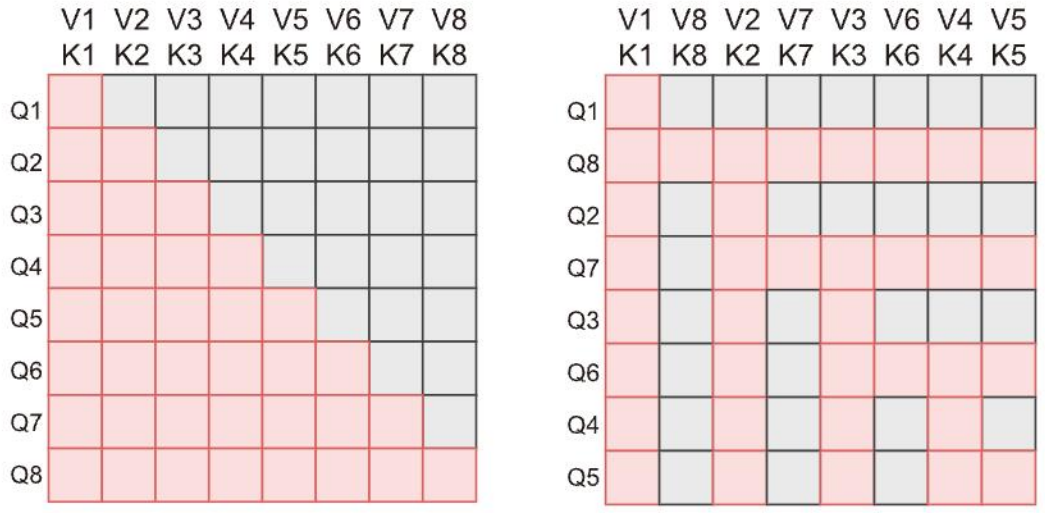
(c) 第 2 步　已看过一半的 KV



(d) 第 3 步　$\boldsymbol{Q}_i$ 看过全部 KV，输出完整

图 《大语言模型：训练、精调与对齐》图 9.24

负载不均与序列置换



(a) 原始切分　　(b) 计算均衡

图 9.25　环形注意力计算负载均衡

对于使用因果掩码的自回归训练，各 GPU 的块注意力计算量是不同的，这可能导致计算负载不均衡和硬件利用率 MFU 下降。

- 原始切分下，$(\boldsymbol{Q}_1, \boldsymbol{Q}_2)$ 块与 $(\boldsymbol{Q}_7, \boldsymbol{Q}_8)$ 块的注意力计算量显著不同：靠后的块能看到的键更多。
- 采用序列置换策略后，$(\boldsymbol{Q}_1, \boldsymbol{Q}_8)$ 与 $(\boldsymbol{Q}_2, \boldsymbol{Q}_7)$ 的注意力计算量一致。
- 从而在整体上，可以减少接近一半的原始块注意力计算量。

实质是让每个设备同时持有一段"靠前"和一段"靠后"的序列，把长短不一的三角形工作量拼成相等的两块。

四种并行的选择

并行方式	切分对象	通信频率	适用场景
数据并行	数据样本	低（每步梯度同步）	通用首选，可跨节点
张量并行	层内参数矩阵	高（前后向均插入）	模型参数更大，需更细粒度切分
流水线并行	层深度	中（阶段边界传激活与梯度）	通信性能较差的场景更有效
上下文并行	序列长度	中（KV 环形传递）	训练序列更长时必需

这些策略可以正交组合，以适应不同规模的模型参数和序列长度需求。当模型参数更大时，需要采用更细粒度的模型并行；当训练序列更长时，则需要引入上下文并行机制。

开放问题

自动并行

如何自动地寻找最优的并行切分策略，是分布式训练系统的关键研究问题之一；同时优化计算效率和减少通信开销也是持续的努力方向。

边界

不减少总计算量

上述并行方法主要解决分布式计算的问题，并未减少模型的总计算量。如何从根源上降低计算量，特别是注意力的平方复杂度，仍是模型设计面临的重大挑战。

9.3

分布式训练系统

本节内容

9.3.1 DeepSpeed 零冗余优化器（ZeRO）

显存占用分析

三个优化阶段

参数切分与流程

ZeRO 用增加通信换取显存：把参数、梯度与优化器状态分区，让每张卡只保留完整状态的 $1/N$。

ZeRO 方法

DeepSpeed 是一个通用的分布式深度学习训练框架，旨在显著降低显存占用。其核心算法是零冗余优化器（Zero Redundancy Optimizer, ZeRO）。

传统数据并行

每个 GPU 设备都存储完整的模型参数、梯度和优化器状态副本。

数据不同、副本相同——同样的状态在 N 张卡上重复了 N 份。

ZeRO

将这些模型状态（权重参数、梯度和优化器状态）划分并分布在集群中可用的 GPU 或 CPU 上。

每张卡只保留自己负责的那一片，需要时再通过通信取回。

> **定位** ZeRO 是对数据并行的一种扩展实现：计算的组织方式仍是数据并行，改变的是状态的存放方式。

模型状态的显存占用



图 9.26　训练过程显存占用

模型状态	精度	字节数
模型参数	FP16	2Ψ
梯度	FP16	2Ψ
优化器状态（FP32 参数副本）	FP32	4Ψ
优化器状态（动量）	FP32	4Ψ
优化器状态（方差）	FP32	4Ψ
合计	—	16Ψ

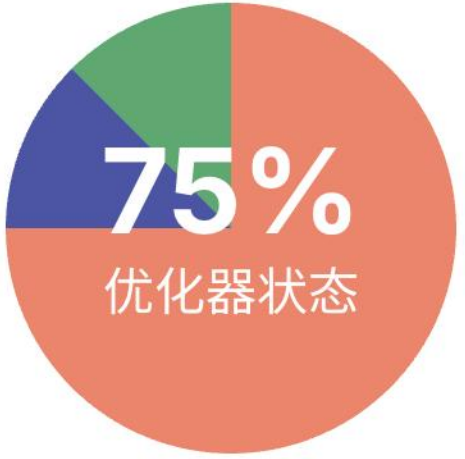
$$2\Psi + 2\Psi + 12\Psi = 16\Psi \text{ 字节}, \qquad 16 \times 7.5 \times 10^9 = 120\ \text{GB}$$

以参数量为 Ψ 的模型配合 Adam 优化器进行混合精度训练为例。7.5B 参数量的模型，其模型状态至少需要 120 GB 显存空间。

除模型状态外，显存还包含激活值、中间计算变量以及显存碎片；中间变量包含通信操作（如 All-Gather）所需的存储开销。数据并行度为 N 时，这些相同的模型状态会在 N 个 GPU 上复制 N 份。

优化器状态的显存占比

在模型状态中，优化器状态通常是显存占用最大的部分。ZeRO 的核心思想之一，正是通过优化器状态分区来消除这部分冗余。



模型状态 16Ψ 的组成（并行度 1）

组成部分	每参数数量	占比
优化器状态	12Ψ	75%
梯度	2Ψ	12.5%
参数	2Ψ	12.5%

120 GB
7.5B 模型的状态总量

75%
优化器状态占比

1.9 GB
ZeRO-3 每卡占用

优化器状态由动量、方差与 FP32 参数副本三份构成；6Ψ 到 16Ψ 的差距全部来自这三份副本。把它们切开，收益远大于切梯度或切参数本身。

依据 《大语言模型：训练、精调与对齐》第 9.3.1 节

ZeRO 的三个阶段

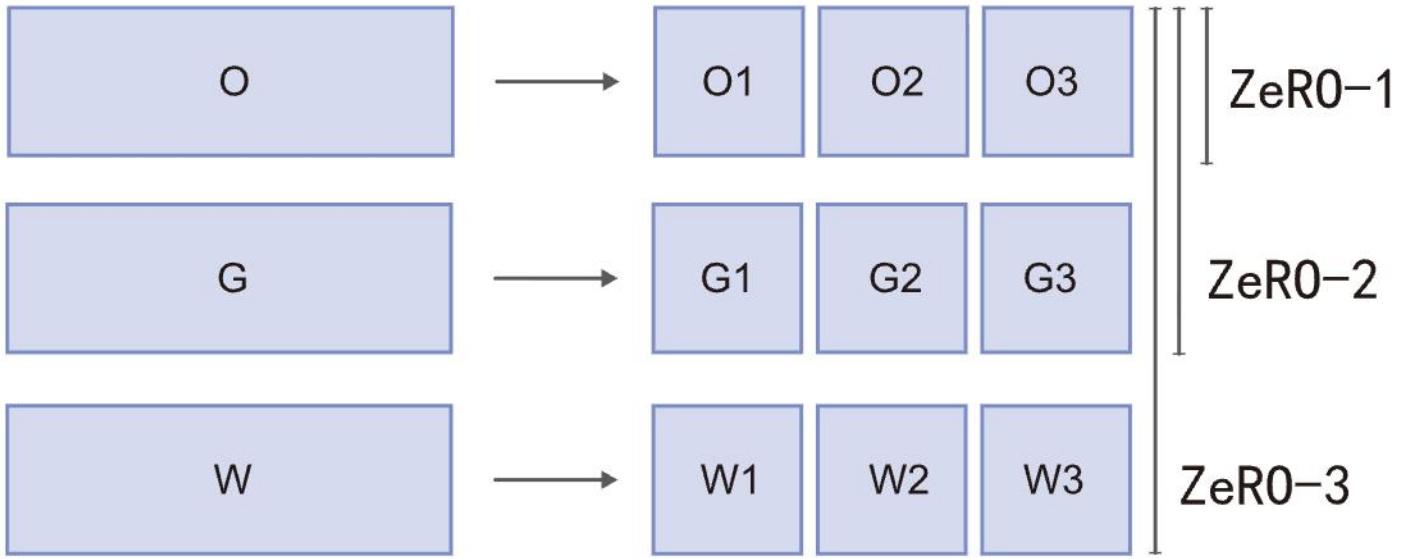


图 9.27　ZeRO 优化的 3 个阶段

- **ZeRO-1**：切分优化器状态，包括动量和方差。
- **ZeRO-2**：在 ZeRO-1 基础上增加梯度切分，确保每个 GPU 上的优化器状态分区与梯度分区相互对应。
- **ZeRO-3**：进一步切分模型参数本身。

需要注意的是，当参数切分未遵循张量并行的行列规则时，无法直接在局部参数分区上执行矩阵乘法运算，需通过通信操作收集参数。

图 《大语言模型：训练、精调与对齐》图 9.27；第 9.3.1 节

ZeRO 三个阶段的通信量

ZeRO 在数据并行基础上引入参数分区机制。应用 ZeRO-3 后，每个 GPU 仅存储完整模型状态的 $1/N$（N 为 GPU 数量），各分区互不重叠，完全消除冗余存储。

阶段	切分内容	每卡状态量	新增通信
基线 DP	不切分	16Ψ	All-Reduce 梯度
ZeRO-1	优化器状态	$4\Psi + 12\Psi/N$	All-Reduce + All-Gather 权重
ZeRO-2	优化器状态 + 梯度	$2\Psi + 14\Psi/N$	Reduce-Scatter + All-Gather
ZeRO-3	参数 + 梯度 + 优化器状态	$16\Psi/N$	前向 All-Gather，反向 Reduce-Scatter

$$\text{每卡状态} = \frac{16\Psi}{N}, \qquad N = 64: \ \frac{120}{64} \approx 1.9\ \text{GB}$$

以 7.5B 参数模型为例：ZeRO-3 阶段每个 GPU 的模型状态显存占用可从 120 GB 降至约 1.9 GB（假设使用 64 个 GPU）。

显存省下来了，但参数不再常驻：每次用到都得先收集，这正是 ZeRO-3 通信开销最大的原因。

ZeRO 三个阶段的显存对比

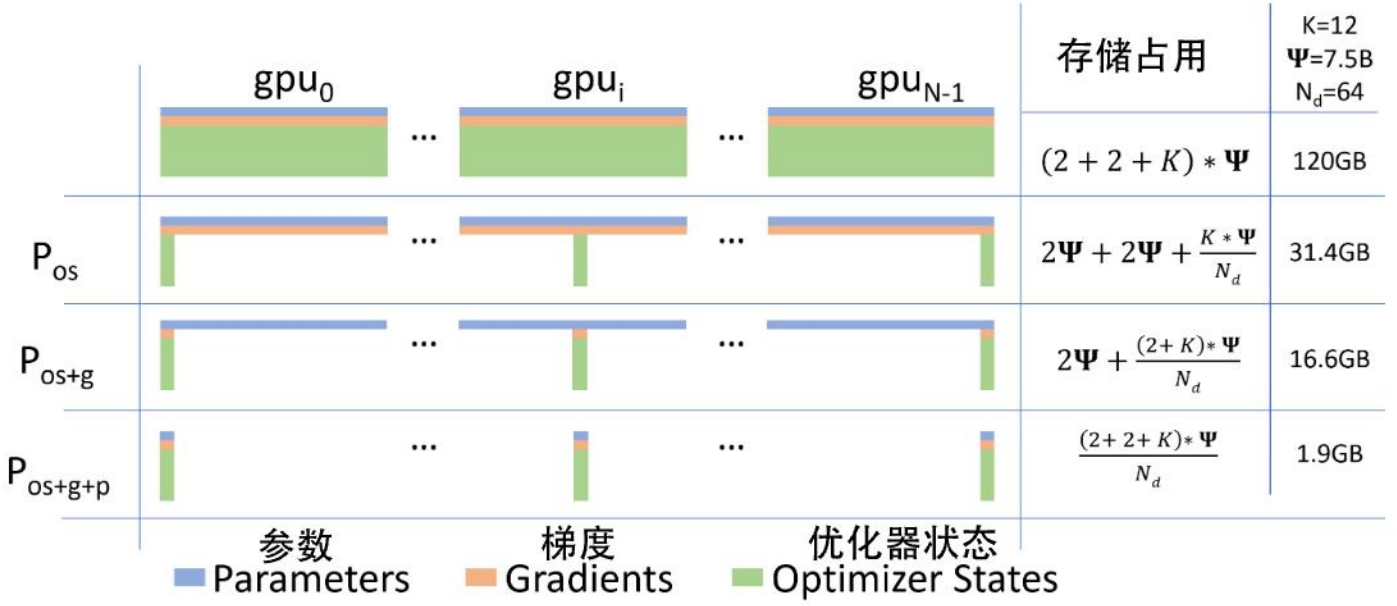
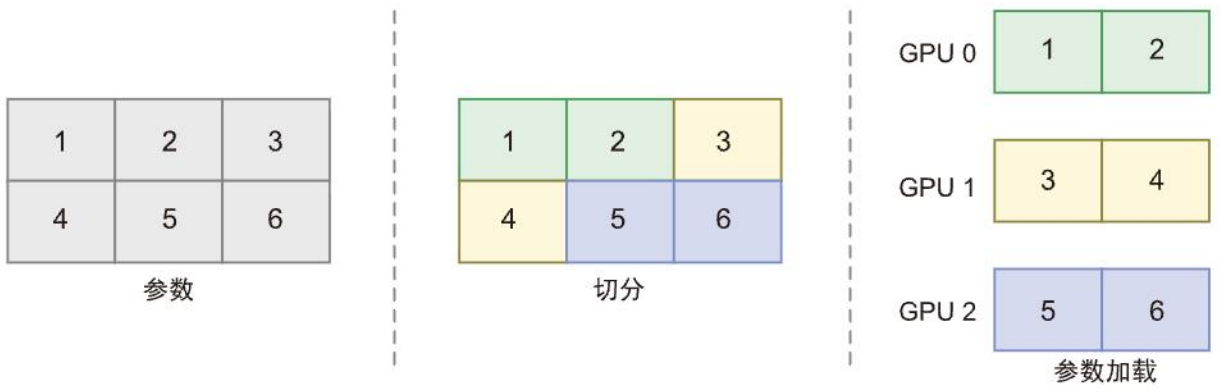


图 9.28　ZeRO 三阶段存储占用分析

图中按 7.5B 参数模型、64 张 GPU 给出各阶段的每卡占用：随切分粒度加深，参数、梯度与优化器状态依次被摊薄，直至全部状态都只保留 $1/N$。

横轴上的每一级都对应一次"多切一类状态、多一类通信"的交换，选择阶段本质上是选择愿意为显存付出多少带宽。

参数的切分方法



① 展平

将多维张量展平为一维向量

② 均分

按预设分区长度均匀切分

③ 记录

记录各分区的起止索引

所有 ZeRO 阶段都需根据参数分区创建对应的优化器状态分区，并维护反向传播产生的梯度分区。

图 9.29　ZeRO 参数切分策略

例如 2×3 大小的参数矩阵在 3 个 GPU 上的切分，按元素序列 $[a_{11}, a_{12}, a_{13}, a_{21}, a_{22}, a_{23}]$ 划分为 3 段，而非按矩阵行、列切分。

灵活性使其适用于任意模型架构，但 ZeRO-3 执行矩阵乘法时需实时收集参数分区（如 All-Gather），引入额外通信。

图 《大语言模型：训练、精调与对齐》图 9.29；第 9.3.1 节

ZeRO-1

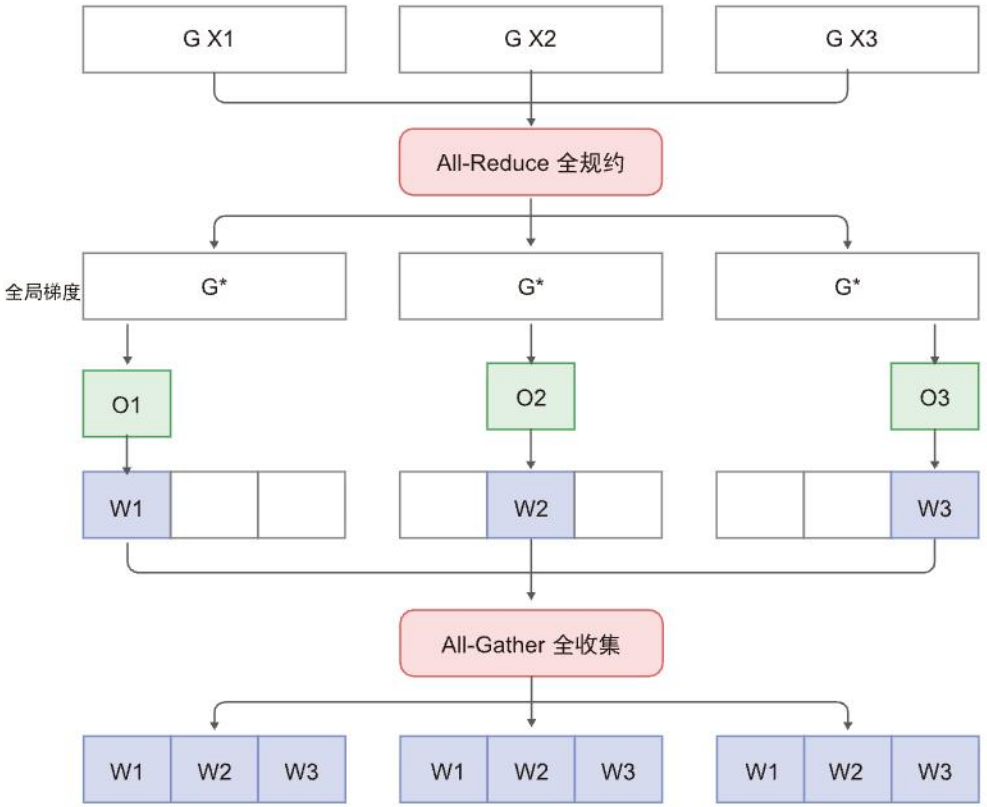


图 9.30　ZeRO-1 优化流程

- 每个 GPU 存储独立的优化器状态分区。
- 数据并行计算后，不同 GPU 上的模型副本产生的梯度不同，需通过 All-Reduce 操作进行均值全规约获得全局梯度。
- 随后，每个 GPU 根据全局梯度更新其本地维护的优化器状态分区所对应的模型权重。

$$\bar{\boldsymbol{g}} = \texttt{All-Reduce}\left(\boldsymbol{g}_0, \cdots, \boldsymbol{g}_{N-1}\right), \quad \boldsymbol{W}^{(i)} \leftarrow \boldsymbol{W}^{(i)} - \eta\, \bar{\boldsymbol{g}}^{(i)}$$

上标 (i) 表示第 i 个 GPU 负责的权重分区；全局梯度在每张卡上都相同，但只有对应分区的权重被更新。

更新完成后，通过 All-Gather 操作收集所有 GPU 上的权重分区，实现权重的一致性更新（见图 9.31）。

图 《大语言模型：训练、精调与对齐》图 9.30；第 9.3.1 节

全收集过程

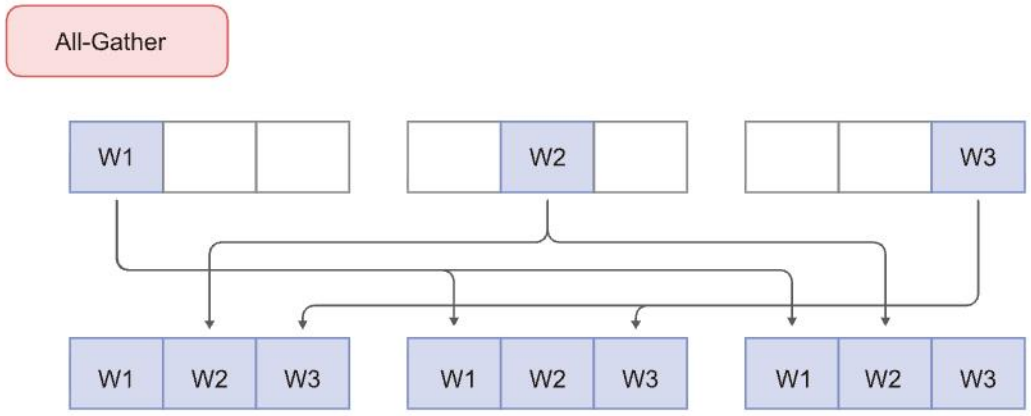


图 9.31　全收集通信算子

分区更新的好处是每个 GPU 只需为自己的那一片维护 FP32 优化器状态；代价是更新后每张卡只有部分权重是最新的。

$$\boldsymbol{W} \leftarrow \texttt{All-Gather}\left(\boldsymbol{W}^{(0)}, \boldsymbol{W}^{(1)}, \cdots, \boldsymbol{W}^{(N-1)}\right)$$

All-Gather 让所有设备重新获得完整的参数副本，为下一轮前向计算做好准备。

这次 All-Gather 是 ZeRO-1 相对普通数据并行新增的通信；到 ZeRO-3，参数本身就分区存放，反而不再需要这一步“额外”的同步。

ZeRO-2

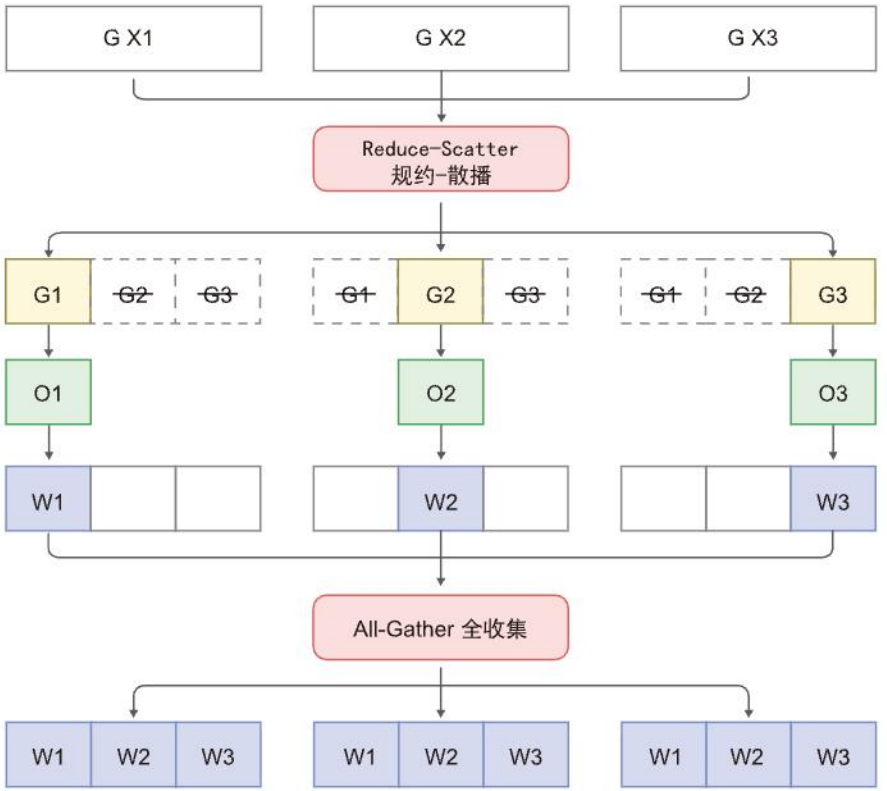


图 9.32　ZeRO-2 优化流程

ZeRO-1 的问题

梯度在各 GPU 上仍持有完整的副本，存在冗余。

ZeRO-2 的做法

将梯度全规约操作替换为规约-散播（Reduce-Scatter），使每个 GPU 仅获得并维护与其本地优化器状态分区相对应的部分梯度。

$$\hat{\boldsymbol{g}}^{(i)} = \texttt{Reduce-Scatter}\left(\boldsymbol{g}_0, \boldsymbol{g}_1, \cdots, \boldsymbol{g}_{N-1}\right)$$

设备 i 只拿回全局梯度的第 i 个分块 $\hat{\boldsymbol{g}}^{(i)}$，正好与它负责的优化器状态分区对应。

进一步分析，训练时通常会使用梯度累计技巧：ZeRO-1 在梯度累计步数里仅在最后一步做梯度全规约操作，ZeRO-2 在每个梯度累计步里都要做规约-散播操作。

ZeRO-3 的前向计算

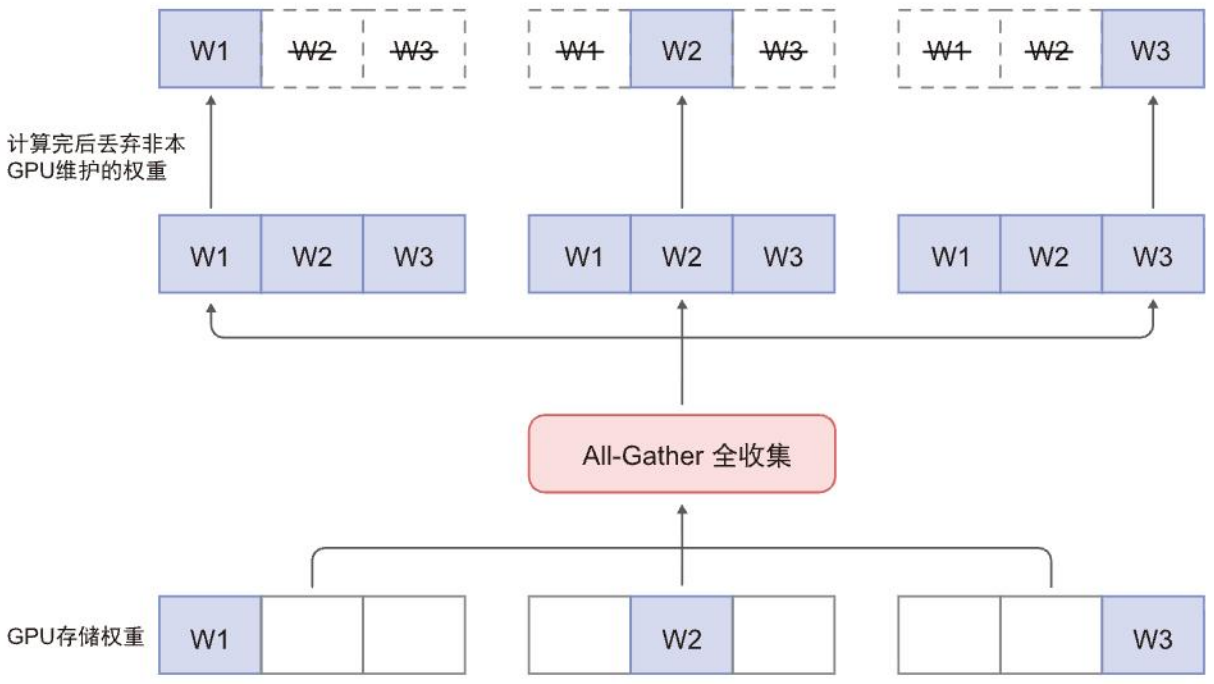


图 9.33　ZeRO-3 前向计算流程

- 前向计算时，每个 GPU 需通过 All-Gather 操作获取完整参数进行计算。
- 计算完成后释放非本地维护的参数分区，显存回到只存 $1/N$ 的状态。
- 参数按层“即取即用”，因此收集与释放的频率与层数成正比。

这一步是 ZeRO-3 最显著的额外开销：矩阵乘法无法在展平后的局部元素上完成，必须先拼接回完整张量。

反向传播时，梯度更新沿用 ZeRO-2 的方式：使用 Reduce-Scatter 操作，使每个 GPU 仅保留其负责维护的梯度分区（见图 9.34）。

图 《大语言模型：训练、精调与对齐》图 9.33；第 9.3.1 节

ZeRO-3 的反向计算



图 9.34　ZeRO-3 反向计算流程

① 前向

All-Gather 取参数，用完释放

② 反向

Reduce-Scatter 得本地梯度分区

③ 更新

就地更新本地参数分区，无需额外同步

参数已分区存储且更新在对应分区上进行，因此无需像 ZeRO-1 那样额外 All-Gather 同步权重。

> **小结**　ZeRO 以通信时间换显存空间；后续版本支持把部分状态卸载至 CPU 内存或外部存储。

图 《大语言模型：训练、精调与对齐》图 9.34；第 9.3.1 节

ZeRO 三个阶段的对比

基线

数据并行

每卡存完整参数、梯度、优化器状态。

All-Reduce 梯度

阶段 1

ZeRO-1

切优化器状态（动量与方差）。

All-Reduce + All-Gather 权重

阶段 2

ZeRO-2

再切梯度，与优化器分区对齐。

Reduce-Scatter + All-Gather

阶段 3

ZeRO-3

再切参数本身，全部状态各存 $1/N$。

前向 All-Gather，反向 Reduce-Scatter

显存 ↓，　通信 ↑

每加深一级，就多切一类状态、多一类通信。ZeRO-3 省下的是参数本身的显存，付出的是每次前向都要重新收集参数。

这也解释了为什么章节小结会说：DeepSpeed 是对数据并行的一种扩展实现，由于其切分策略是通用的，在大型模型训练任务上计算效率不如以张量并行为主的 Megatron 框架。

9.4 小结

本节内容

系统设计的三条主线

四种并行算法的定位

成本与迭代速度

并行算法的核心理念，是把计算任务规模均等地划分成子任务，并使计算规模和通信负载同时保持均衡。

训练系统的三条主线

对于大型的分布式训练任务，其训练系统需要综合考虑硬件、模型结构和并行算法等特性，精心设计混合的并行策略，从而最大化利用计算资源并提高训练吞吐。

硬件

节点内带宽、节点间网络、显存容量

模型结构

层数、层内矩阵形状、注意力复杂度

并行算法

DP / TP / PP / CP 的组合与切分因子

吞吐

单位时间内处理更多词元

> **核心理念** 将计算任务规模均等地划分成子任务，并且使得计算规模和通信负载均衡。任何一维切得过细，都会让某一段链路成为新的瓶颈。

四种算法的适用范围

算法	优势	局限与适用条件
数据并行	实现简单、通用，可扩展到大量设备	状态冗余大；需 ZeRO 等技术削减
张量并行	切分层内参数，单卡显存显著下降	通信频繁，宜限制在节点内；推理场景也常用
流水线并行	设备间通信频率更低	存在气泡；通信性能较差的场景更有效
上下文并行	把长序列切到多卡	长文本训练的重要手段，也可加快推理的预填充

DeepSpeed

数据并行的扩展

由于切分策略是通用的，在大型模型训练任务上，计算效率不如以张量并行为主的 Megatron 框架。

上下文并行

可迁移的长序列手段

可以扩展到其他长序列注意力场景；在推理场景中可以加快预填充。

本章详解的四种并行算法构成 4D 并行的四个正交维度，实际系统按瓶颈选择切分哪一维、切多细。

训练效率的意义

大语言模型参数规模的急剧增加，为分布式训练领域带来快速发展的动力。在大型训练任务中，优化训练的成本是经济的；而更重要的是，减少耗时能为模型迭代研发带来竞争力。

成本

单位算力更值钱

集群规模越大、运行越久，MFU 每提高一个百分点所节省的机时与电费都相当可观。

时间

迭代周期缩短

一次预训练以周、月计；缩短耗时就意味着同一时间内能做更多轮实验与验证。

竞争力

研发节奏

训练系统的效率最终转化为模型迭代的速度，这是工程投入最直接的回报。

回看 从 9.1 的显存账本，到 9.2 的四种切分，再到 9.3 的状态分区，本章的每条线索都指向同一个目标：让同样的 GPU 在同样的时间里算出更多的梯度。

9.5

附录：专家并行*

MoE 把前馈层拆成多个专家：每台设备承载不同的专家，词元经由门控网络被派发到对应设备上计算。

本节内容

专家并行的部署方式

派发与合并：两次 All-to-All

负载不均衡与通信重叠

专家并行

在包含 MoE 模块的语言模型中，GShard 提出了专家并行：多设备部署结构相同的网络，但各设备的前馈层专家模块加载不同专家的参数。

- 每个设备处理独立的输入数据批次，因此在设备部署方式上与数据并行类似。
- 但核心差异在于：每个设备承载的是模型中不同的专家模型，而非完整的模型副本。
- 门控网络为每个词元选择 Top-K 个专家，决定了词元要去哪台设备。

对比

与数据并行的差别

数据并行中每张卡持有同一份完整参数；专家并行中每张卡只持有部分专家的参数，其余模块仍可能复制。

因此两种并行的"切分对象"不同：一个是数据，一个是前馈层中的专家集合。

关键 每个词元会通过门控网络被动态路由到 Top-K 个专家——路由结果在运行时才确定，这正是专家并行通信复杂的根源。

专家并行的数据流

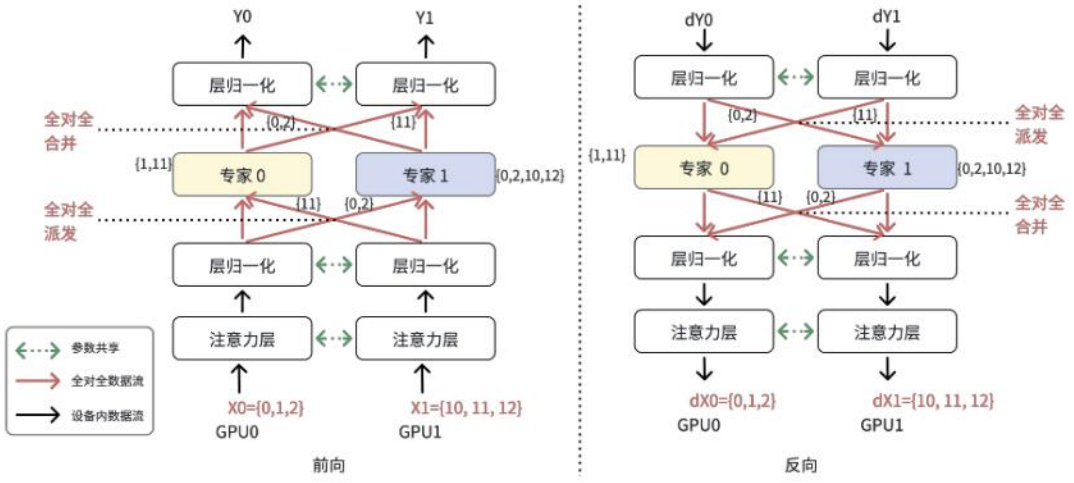


图 9.35　专家并行前后向数据流图

数据流可以概括为“派发—专家计算—合并”：词元先被送到持有目标专家的设备，专家算完后结果再被送回原设备，组合成最终的输出向量。

门控路由
每个词元选 Top-K 个专家

→

派发 Dispatch
All-to-All 送到专家所在设备

→

专家计算
各设备并行处理本地词元

→

合并 Combine
All-to-All 送回原设备求和

专家并行的两次全交换

这涉及两次关键的 All-to-All 通信操作：首先将词元派发到其目标专家所在的设备，然后在各专家处理完其分配到的词元后，将计算结果合并回原始设备。

阶段	通信原语	数据内容
派发 Dispatch	All-to-All	按目标专家分组后的词元隐藏状态
专家计算	无通信	各设备在本地词元上执行前馈层
合并 Combine	All-to-All	专家输出按来源设备回送并加权求和

$$\boldsymbol{y}_t = \sum_{k \in \text{Top-}K(t)} g_{t,k} \cdot \mathbf{Expert}_k\left(\boldsymbol{x}_t\right)$$

$g_{t,k}$ 为门控网络给出的权重。词元 t 只由被选中的 K 个专家计算，其余专家不参与，因此每台设备的工作量取决于被路由到它的词元数量。

门控路由导致的负载不均衡（不同专家分配到的词元数量不同）会直接引发通信量不均衡和各设备计算量不均衡的问题。

DualPipe 与通信重叠

为缓解专家并行中通信开销的影响，DeepSeek-V3 在其 MoE 模块中设计了双向流水线和计算-通信重叠机制，以更高效地隐藏通信延迟。

机制 1

双向流水线

从两个方向同时填充流水线，使同一时刻既有前向也有反向的计算在进行，从而压缩气泡。

机制 2

计算-通信重叠

把 All-to-All 的派发与合并安排在计算进行的同时，用计算时间掩盖通信时间。

> **同一思路** 9.1.3 节把“通信导致设备空闲”列为降低 MFU 的主因；这里给出的解法与流水线的 1F1B、1F1B1W 完全一致——不是消除通信，而是让它与计算并行。

双向流水线把前向与反向分到两个方向上同时填充，使“升温”与“收尾”两个气泡互相抵消；计算-通信重叠把 All-to-All 切成小块，与专家计算交替推进。两者叠加后，流水线在大部分时间保持满载。

结合 8 位精度训练等技术，DeepSeek-V3 在大型模型上进一步降低了计算量与通信量，说明显存、带宽与数值精度三者可以同时优化。

9.6

附录：训练-推理解耦架构*

PPO、GRPO 等算法需要在线采样：把推理集群与训练集群分开部署，才能让采样与训练各自跑在合适的并行策略上。

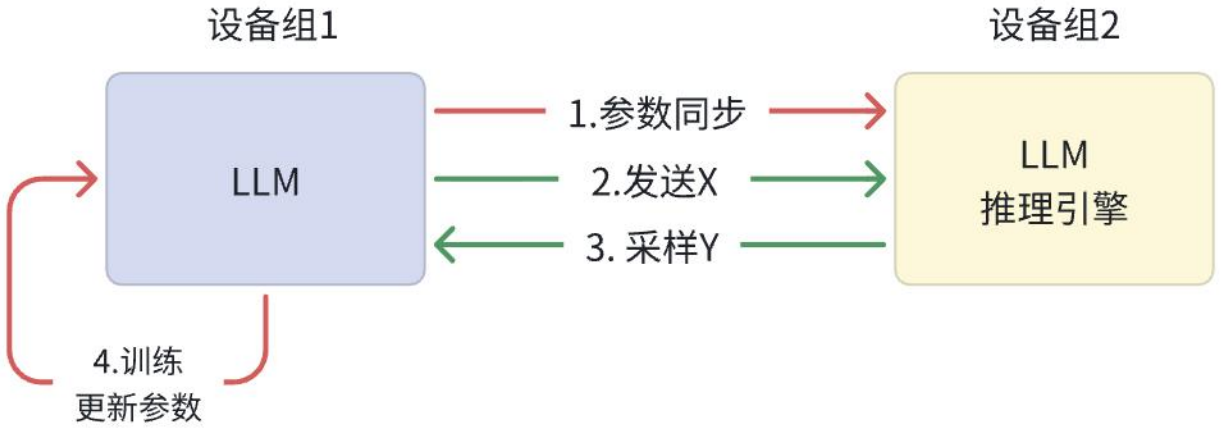
本节内容

解耦架构的动机

参数同步的开销

异步推理与离轨策略

训练与推理解耦的原因



PPO 和 GRPO 等算法需要与环境在线交互。为此可采用训练-推理解耦架构：由专门的推理集群负责高效的数据采样。

推理集群

负责采样

与环境和提示词交互，生成 rollout；推理侧偏好低延迟、大并发的批处理。

训练集群

负责更新

在采样数据上计算策略梯度并更新参数；训练侧偏好高吞吐、状态分片。

图 9.36　训练-推理解耦架构示意图

解耦架构的参数同步

推理集群的参数需及时更新为训练产生的最新参数，以保证采样有效性，但这引入了参数同步开销。

训练更新
产生新参数 θ_{t+1}

→

参数同步
推送权重到推理集群

→

采样
用 θ_{t+1} 生成新轨迹

→

训练
用新轨迹算梯度

> **代价** 由于参数同步存在耗时，训练服务器的计算资源在等待推理结果时可能处于闲置状态。

这实质上是把数据并行里的“梯度同步”换成了“参数同步”：同步的数据量从梯度变成完整权重，频率也从每步一次变成每轮采样一次。此类架构的实现可参考 OpenRLHF 或 verl。

异步推理

为了提高系统整体的并行性与资源利用率，可以设计异步推理流程，例如 AReaL 所提出的方案。

同步推理

训练必须等待当前策略的采样完成，参数版本一致。

数据最新，但训练侧存在空等。

异步推理

训练与采样并行推进，训练使用稍早策略生成的数据。

利用率高，但数据滞后于当前策略。

> **算法要求** 在此异步模式下，训练过程使用的数据可能滞后于当前策略，因此算法需采用离轨策略优化方法进行训练。

进一步推广到更复杂的智能体系统，智能体与环境交互时可能涉及搜索网页、下载数据、运行代码等开放世界操作，单次交互的响应时间显著延长。这使得设计高效的异步推理机制，以及适配长延迟环境的离线策略训练算法，变得尤为关键。

思考题与实践题

下面 8 题均为动手实现，建议先用 2–4 个进程在本机跑通，再放大到多卡。实现时不必追求与生产框架同等的性能，重点是把通信原语的语义与数值一致性验证清楚。

1 实现 Reduce-Scatter：输入各进程的梯度张量，输出按 rank 分块规约的结果，并与 All-Reduce 后切片的结果逐元素比对。

2 实现非阻塞的 All-to-All：用 `isend/irecv` 拼出不带全局屏障的交换，比较它与阻塞版本在多轮叠加下的耗时。

3 实现分布式 Adam 优化器及梯度累积更新：参数与优化器状态分区存放，验证累积 K 步后更新与逐步更新等价。

4 实现 GQA 的张量并行前向与反向：KV 头在 TP 组内共享，验证前向输出与单卡结果一致、反向梯度能正确规约。

5 实现流水线并行的 1F1B：P 个阶段、M 个微批量，画出各阶段的时隙表，标出气泡并计算气泡占比。

6 实现基于 All-Reduce 的词表并行：输出层按词表切分，前向 All-Gather、反向 Reduce-Scatter，验证与不分词的 logits 一致；再实现嵌入层词表并行，嵌入表按词表切分，前向 All-Gather 拼出各词元向量、反向 Reduce-Scatter 分回梯度。

7 实现 ZeRO-3 训练：参数、梯度、优化器状态三分区，前向 All-Gather 取参数、用完释放，报告每卡显存与通信量。

8 实现专家并行，再进一步实现计算-通信重叠：用异步的 All-to-All 与专家计算重叠，比较重叠前后的吞吐率。

推荐使用 torch.distributed 模块进行实现。